Mattermost Cross Guard

Cross-domain message relay for Mattermost Federal

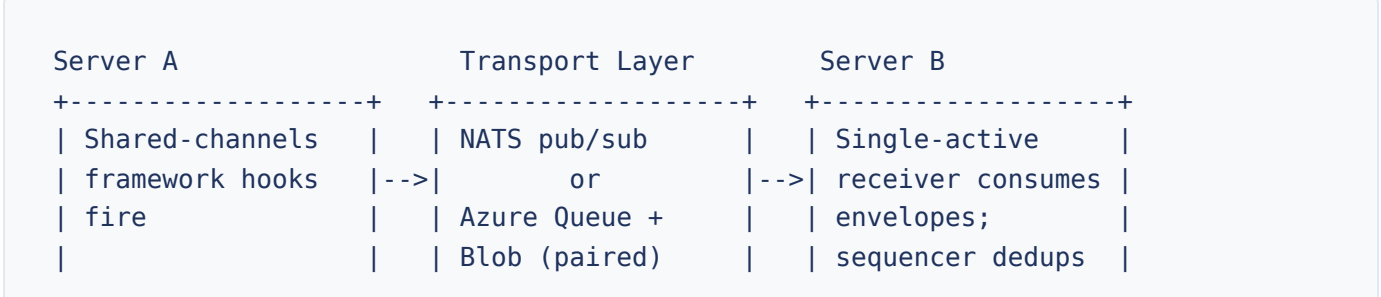
What is Cross Guard?

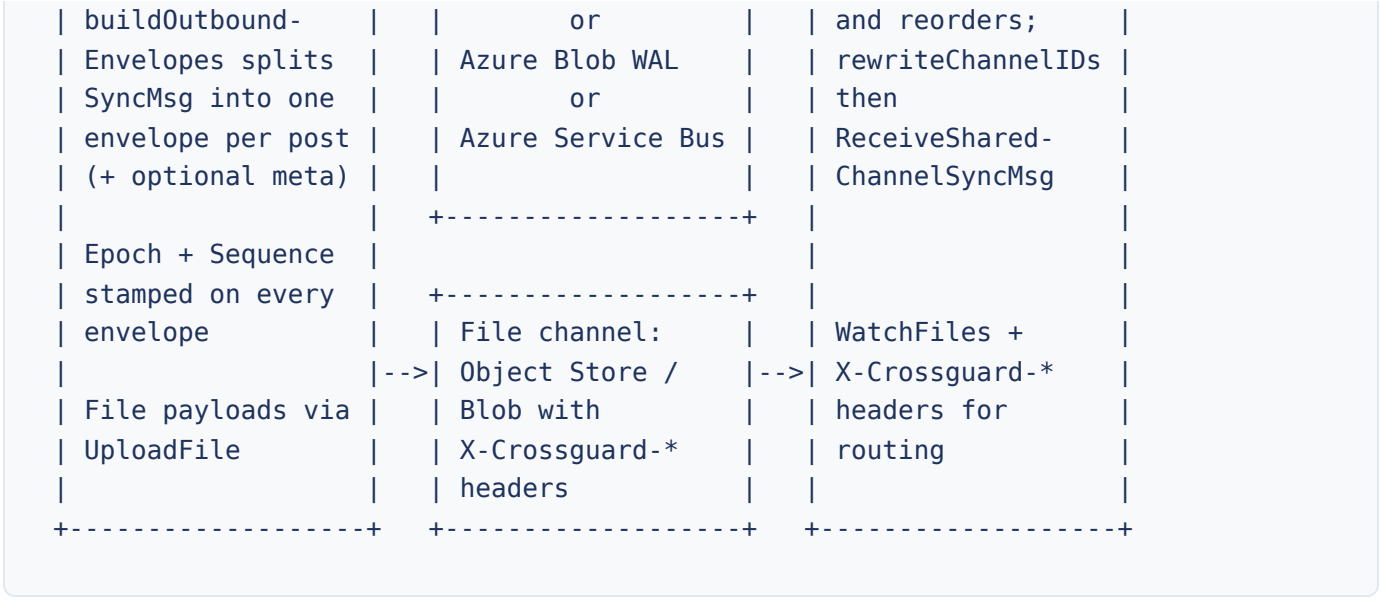
Cross Guard enables message relay between separate Mattermost servers using one of four pluggable transport providers: NATS, Azure Queue Storage, Azure Blob Storage, or Azure Service Bus. It is designed for environments where multiple isolated Mattermost instances need to exchange messages across domain boundaries without direct database or API access between servers.

Posts, edits, deletes, reactions, channel join and leave events, post acknowledgements, user identity records, and file attachments flow bidirectionally across server boundaries. Each server maintains its own users, teams, and channels. Cross Guard bridges them at the message level, preserving each server's independence while enabling real-time communication. File attachments and profile images are transferred through the configured transport's file-upload channel (NATS JetStream Object Store, Azure Blob Storage, or the Service Bus blob sidecar, depending on provider). File transfer is opt-in per connection.

The plugin integrates with Mattermost's shared-channels framework: outbound content is collected by the framework's sync hooks (`OnSharedChannelsSyncMsg` and its companions), and inbound content is handed back to the framework via `ReceiveSharedChannelSyncMsg`. Relay behaviour is therefore transparent to end users; messages appear in channels as if they were posted locally, attributed to a synced remote user that the webapp's user popover marks as "Relayed from".

How Federation Works





Two Connection Directions

- **Outbound:** The plugin implements the shared-channels framework hooks (`OnSharedChannelsSyncMsg`, `OnSharedChannelsAttachmentSyncMsg`, `OnSharedChannelsProfileImageSyncMsg`, `OnSharedChannelsPing`) and fans each upstream sync cycle into one envelope per Post, plus an optional metadata envelope for non-post content. Envelopes are serialized as XML by default (JSON support is in the immediate follow-up), Epoch / Sequence-stamped, and published via the configured transport. File attachments and profile images ride a parallel file-upload channel.
- **Inbound:** Exactly one cluster node holds the active subscription for a given inbound connection at a time (KV TTL lease, 45 s TTL with 15 s renewal). Incoming bytes are deserialized (format auto-detected from the first non-whitespace byte), admitted by the inbound sequencer (`(Epoch, ConnName, ChannelId, Sequence)` dedup, bounded reorder, audited gap-deadline), translated through the channel-ID rewrite, and handed to the framework's `ReceiveSharedChannelSyncMsg` for application.

What Gets Relayed

Each `sync_msg` envelope carries a payload whose components are pulled from upstream `mmModel` types into plugin-owned wire types (under `server/wire/`) with source-only and receiver-overwritten fields dropped at the boundary.

Payload component	Carried?	Notes
Posts (new, edited, deleted)	Yes	One Post per envelope. Edits signalled by `EditAt` / `OriginalId`; deletes by a non-zero `DeleteAt` and an optional `Props.DeleteBy`. Threading preserved via `RootId`.
Users	Yes	Whitelisted subset of `mmModel.User` (identity, names, position, roles, locale, optional Timezone, four-key `UserProps`). Credentials, MFA, NotifyProps dropped at the boundary.
Reactions	Yes	Emoji name plus author. Reaction removals carry a non-zero `DeleteAt`.
Presence / Status	Yes	Online / away / dnd / offline with manual flag and DND end time. Source-local `ActiveChannel` is dropped.
Membership changes	Yes	Channel join / leave events relayed so the framework keeps its synced-channel membership view current.
Post acknowledgements	Yes	The "ack" feature's user-by-user ack timestamps.
Mention transforms	Yes	Sender-resolved mention rewrites (e.g. `@alice` → `@alice.high`) so receivers reproduce the source author's intent.
File attachments	Configurable	Opt-in per connection via `file_transfer_enabled`. Carried on the file-upload channel (not the XML envelope); supports allow / deny extension filters.
Profile images	Configurable	Carried on the same file-upload channel when file transfer is enabled.
Whitelisted Props keys	Yes	Fifteen keys on `PostProps`, four on `UserProps`. Unknown upstream keys (from any plugin) are dropped at the boundary.
Loop prevention	(handled)	Intrinsic to the framework: every received User carries a non-empty `RemoteId`, so the framework's cursors do not re-emit synced content. The plugin additionally calls

Payload component	Carried?	Notes
		`connNameForRemote(rc.RemoteId)` to suppress per-remote re-emission.

Key Concepts

Connections

Named connection configurations (NATS, Azure Queue Storage, Azure Blob Storage, or Azure Service Bus), inbound or outbound, defined in the System Console. Referenced by name in all commands using the format `direction:name` (e.g. `outbound:relay-to-b`).

Team Initialization

A team must be linked to a connection before any channels in that team can relay messages. This is always the first step. Use `/crossguard init-team` to link a connection.

Channel Initialization

Individual channels within an initialized team are linked to connections. Only linked channels relay messages. Use `/crossguard init-channel` to link a channel.

Inbound Prompts

When an inbound envelope arrives for a team or channel that has not been explicitly accepted, an interactive prompt (Accept / Block buttons) is posted. Team-level prompts appear in town-square; channel-level prompts appear in the target channel. Admins must accept before messages flow. Use `/crossguard reset-prompt` or `/crossguard reset-channel-prompt` to clear a blocked prompt.

Team Name Rewriting

Maps remote team names to local teams, allowing servers with different team names to relay correctly. Configured per inbound connection using `/crossguard rewrite-team`.

Synced Remote Users

Inbound users from a remote server are upserted by the shared-channels framework as *remote users*: local accounts that carry a non-empty `RemoteId` identifying the source remote. They are distinct from native local users and cannot silently merge into existing local accounts. The webapp's user popover shows a "Relayed from: alice (via high-side)" indicator on synced users so operators always know which conversation came from which domain.

Single-Active Inbound Receiver

For each inbound connection, exactly one node in a Mattermost cluster holds the active subscription at a time, gated by a KV TTL lease (45 s TTL with 15 s renewal). Other nodes idle and race for the lease when it ages out. The inbound sequencer keys on `(Epoch, ConnName, ChannelId, Sequence)` to absorb at-least-once redeliveries from the transport and replays caused by a sender restart; lost ranges are audited via numeric error codes (see Error Codes).

Request-Mode Workflows

In deployments where `RestrictToSystemAdmins` is enabled, two opt-in settings restore a request-and-approve flow so non-sysadmin admins can still drive cross-domain setup without unsupervised linking authority:

- **`AllowTeamAdminRequests`**: a Team Admin running `init-team` submits a link request to System Admins, who approve or deny via interactive DM. `teardown-team` remains direct.
- **`AllowChannelAdminRequests`**: a Channel Admin running `init-channel` submits a link request to Team Admins, who approve or deny via interactive DM. `teardown-channel` remains direct.

Getting Started

1. **Configure connections** in **System Console > Plugins > Cross Guard**. Add connection details for your chosen provider (NATS, Azure Queue Storage, Azure Blob Storage, or Azure Service Bus). See the Admin Configuration Guide for details.

2. **Initialize a team:** Run `/crossguard init-team [connection-name]` in the team you want to enable.

3. **Initialize channels:** Run `/crossguard init-channel [connection-name]` in each channel that should relay messages.

4. **Verify:** Run `/crossguard status` to confirm the setup and check that connections are active.

Permission Model

Role	Scope	Notes
SYSTEM ADMIN	All commands, all teams/channels. Can configure connections in System Console. Can test connections via API. Sees global status overview.	Always has full access regardless of `RestrictToSystemAdmins` setting.
TEAM ADMIN	`init-team`, `teardown-team`, `init-channel`, `teardown-channel`, `reset-prompt`, `reset-channel-prompt`, `rewrite-team`, `status` within their team.	Access blocked when `RestrictToSystemAdmins` is enabled, unless `AllowTeamAdminRequests` is also enabled (see Request Mode below).
CHANNEL ADMIN	`init-channel`, `teardown-channel` within their channel (team must already be initialized).	Access blocked when `RestrictToSystemAdmins` is enabled, unless `AllowChannelAdminRequests` is also enabled (see Channel Request Mode below).
MEMBER	`status` (team-scoped view), `help`.	Can see which connections are active for their team/channel.

> **Note**
> When `RestrictToSystemAdmins` is enabled in plugin settings, Team Admins and Channel Admins lose access to connection management commands. Only System Admins can manage connections.

> **Request Mode (Team)**

When both `RestrictToSystemAdmins` and `AllowTeamAdminRequests` are enabled, Team Admins regain limited access: `init-team` submits a link request for System Admin approval (instead of linking directly), and `teardown-team` allows direct unlinking without approval. System Admins receive request DMs with interactive Approve/Deny buttons. See the Admin Configuration page for details.

Channel Request Mode
When both `RestrictToSystemAdmins` and `AllowChannelAdminRequests` are enabled, Channel Admins regain limited access: `init-channel` submits a channel link request for Team Admin approval (instead of linking directly), and `teardown-channel` allows direct unlinking without approval. Team Admins receive request DMs with interactive Approve/Deny buttons. See the Admin Configuration page for details.

Documentation Pages

Slash Command Reference

Complete documentation for all 9 slash commands, including arguments, permissions, and examples.

Admin Configuration Guide

Connection setup for NATS and Azure, relay settings, example configurations, and troubleshooting.

REST API Reference

All REST endpoints with request and response examples and authentication details.

Error Codes

Every numeric `error_code` emitted in Cross Guard logs, grouped by source file, with descriptions and troubleshooting steps.

Transport Interface

Wire protocol reference: message envelope, payload schemas, connection configuration for NATS and Azure Queue Storage, and complete examples.

Threat Model

STRIDE threat analysis, trust zones, data flow documentation, and CDS deployment recommendations.

CDS Whitepaper

Comprehensive architecture, deployment topologies, security analysis, and cross-domain evaluation guide.

Slash Command Reference

All commands for managing Cross Guard relay connections

Overview

- All commands use the trigger `/crossguard`
- Autocomplete is enabled (hints appear as you type)
- Commands return ephemeral responses (only visible to the user who ran them)
- When multiple connections exist and no name is specified, an interactive selection dialog appears

Quick Reference

Command	Description	Permission	Arguments
`init-team [name]`	Link connection to team	TEAM ADMIN+	Optional connection name
`init-channel [name]`	Link connection to channel	CHANNEL ADMIN+	Optional connection name
`teardown-team [name]`	Unlink connection from team	TEAM ADMIN+	Optional connection name
`teardown-channel [name]`	Unlink connection from channel	CHANNEL ADMIN+	Optional connection name
`reset-prompt <name>`	Clear team connection prompt	TEAM ADMIN+	Required connection name
`reset-channel-prompt <name>`	Clear channel connection prompt	TEAM ADMIN+	Required connection name

Command	Description	Permission	Arguments
`rewrite-team [name] [team]`	Set/clear team name rewrite	TEAM ADMIN+	See details
`status`	Show relay status	ANY MEMBER	None
`help`	Show help	ANY MEMBER	None

/crossguard init-team [connection-name]

Link a connection to the current team, enabling cross-domain relay for this team.

Permission	Team Admin or System Admin (blocked by `RestrictToSystemAdmins`; see Request Mode below)
Arguments	`connection-name` (optional). The display key of the connection (e.g., `outbound:relay-to-b`).

Behavior

- If omitted and exactly one connection exists in config, auto-selects it
- If omitted and multiple connections exist, opens an interactive selection dialog with all configured connections
- If specified but not found, returns error listing available connections
- If connection already linked to this team, returns "already linked" message
- On first connection linked to team, adds team to initialized teams list
- Posts announcement in `#town-square`: "Cross Guard connection `outbound:relay-to-b` linked to this team by @username"

> **Request Mode**
> When both `RestrictToSystemAdmins` and `AllowTeamAdminRequests` are enabled, Team Admins who run this command submit a **link request** instead of linking directly. All System Admins receive a DM with Approve/Deny buttons. The requester is notified of the outcome. If a request is already pending for the same team and connection, the command returns an error.

Examples

```
/crossguard init-team
/crossguard init-team outbound:relay-to-b
```

Error Cases

- `No connections configured. Add connections in the System Console first.`
- `Connection not found: xyz. Available connections: outbound:relay-to-b, inbound:relay-from-a`
- `a request is already pending for this connection` (request mode only)

/crossguard init-channel [connection-name]

Link a connection to the current channel. The team must be initialized first with the same connection.

Permission	Channel Admin, Team Admin, or System Admin (blocked by `RestrictToSystemAdmins`; see Channel Request Mode below)
Arguments	`connection-name` (optional, same auto-select/dialog behavior as init-team)

Behavior

- Validates team is initialized first. If the team does not yet have this specific connection linked, the REST API auto-links the team. The slash command requires explicit team initialization via `/crossguard init-team`.
- If connection not linked at team level, returns: `Connection X is not linked to this team. Run /crossguard init-team first.`
- Adds link emoji prefix to channel header (visible indicator)
- Publishes WebSocket event `channel_connections_updated` to update UI indicators
- Posts announcement in the channel

> **Channel Request Mode**
> When both `RestrictToSystemAdmins` and `AllowChannelAdminRequests` are enabled, Channel Admins can run this command to submit a channel connection link request. The request is sent as a DM

to all Team Admins with **Approve** and **Deny** buttons. The channel is not linked until a Team Admin approves the request. Team Admins and System Admins still link channels directly.

Examples

```
/crossguard init-channel
/crossguard init-channel outbound:relay-to-b
```

/crossguard teardown-team [connection-name]

Unlink a connection from the current team.

Permission	Team Admin or System Admin (blocked by `RestrictToSystemAdmins`; see note below)
Arguments	`connection-name` (optional). If omitted and multiple connections linked, opens dialog. If one linked, auto-selects.

Behavior

- If no connections linked: `No connections are linked to this team.`
- If this was the last connection, removes team from initialized teams list
- Posts announcement in `#town-square`
- If a pending connection link request exists for the same team and connection, the request is automatically cancelled and the requester is notified

> **Request Mode**
> When both `RestrictToSystemAdmins` and `AllowTeamAdminRequests` are enabled, Team Admins can run this command to unlink connections directly (no approval needed for unlinking). This is the only connection management action Team Admins can perform without System Admin approval in request mode.

> **Warning**
> This command does NOT automatically teardown channel connections. Channels that still have this connection linked become orphaned. Run `/crossguard teardown-channel` in affected channels

first.

Example

```
/crossguard teardown-team outbound:relay-to-b
```

/crossguard teardown-channel [connection-name]

Unlink a connection from the current channel.

Permission	Channel Admin, Team Admin, or System Admin (blocked by `RestrictToSystemAdmins`; Channel Admins allowed in channel request mode)
Arguments	`connection-name` (optional, same auto-select/dialog behavior)

Behavior

- If no connections linked: `No connections are linked to this channel.`
- If this was the last connection on the channel, removes link emoji prefix from channel header
- Publishes WebSocket event to update UI indicators
- Posts announcement: "All relays for this channel are now inactive."
- If a pending channel connection request exists for the same channel and connection, the request is automatically cancelled and the requester is notified

> **Channel Request Mode**
> When both `RestrictToSystemAdmins` and `AllowChannelAdminRequests` are enabled, Channel Admins can run this command to unlink channel connections directly (no approval needed for unlinking). This is the only channel connection management action Channel Admins can perform without Team Admin approval in channel request mode.

/crossguard reset-prompt <connection-name>

Clear a blocked or pending inbound connection prompt for the current team. After clearing, a new prompt will appear on the next inbound message.

Permission	Team Admin or System Admin
Arguments	`connection-name` (required)

Behavior

- Looks up prompt in KV store for this team + connection name
- If no prompt found: `No prompt found for connection X on this team.`
- Deletes the prompt entry, allowing the system to create a fresh prompt on the next inbound message

Use Case

An admin accidentally blocked an inbound connection and wants to re-evaluate it.

Example

```
/crossguard reset-prompt relay-from-a
```

/crossguard reset-channel-prompt <connection-name>

Clear a blocked or pending inbound connection prompt for the current channel. Same behavior as `reset-prompt` but operates on channel-level prompts.

Permission	Team Admin or System Admin
Arguments	`connection-name` (required)

Example

```
/crossguard reset-channel-prompt relay-from-a
```

/crossguard rewrite-team [connection-name] [remote-team-name]

Set or clear a remote team name rewrite for an inbound connection. When set, inbound messages with the specified remote team name are routed to this team instead of requiring a matching local team name.

Permission	Team Admin or System Admin
Arguments	`connection-name` (required if multiple inbound connections linked, optional if exactly one). `remote-team-name` (optional; omit to clear the rewrite).

Behavior

- Only works with inbound connections. If no inbound connections linked: `No inbound connections are linked to this team.`
- Sets a rewrite index in the KV store: `connection-name + remote-team-name -> local team ID`
- Returns HTTP 409 (Conflict) if the rewrite index already exists for another team
- Posts audit message in `#town-square` showing who made the change
- When clearing: removes the rewrite index entry and clears `RemoteTeamName` from the team connection metadata

Examples

```
/crossguard rewrite-team relay-from-a remote-team-alpha
/crossguard rewrite-team relay-from-a
```

/crossguard status

Show Cross Guard initialization status for the current team and channel.

Permission	Any team member
Arguments	None

Output Varies by Role

- **Regular users / Team Admins:** Current channel status (relay enabled yes/no), current team status (initialized yes/no), team connections list, channel connections list. Each connection displays file transfer status (e.g., `files off`, `files on`, `files on, allow: .pdf,.docx`).
- **System Admins:** Global overview with all initialized teams (table with display name, team ID, team name, linked connections), all configured connections (table with name, direction, address, auth type, subject, file transfer status), plus current channel status. Warnings shown if connection configuration cannot be parsed.

/crossguard help

Show inline help for all commands. No arguments or special permissions required.

Connection Selection Dialog

When a command needs a connection name and multiple connections are available, an interactive dialog opens automatically.

- Dialog title varies: "Link Connection to Team", "Unlink Connection from Team", etc.
- Dropdown shows all available connections in `direction:name` format
- Dialog submits to `/plugins/crossguard/api/v1/dialog/select-connection`
- State encodes the action and target: `init-team:{teamID}`, `init-channel:{channelID}`, etc.

Admin Configuration Guide

Transport connections, relay settings, and deployment examples

Accessing Plugin Settings

Navigate to **System Console > Plugins > Cross Guard**. The plugin must be enabled before configuration is accessible. Settings are organized into two sections: Connection Configuration and Relay Settings.

Connection Configuration

Understanding Inbound vs Outbound

Direction	Flow	Use When
Outbound	Mattermost → Transport	Your server is the **sender**. Publishes messages to the configured transport.
Inbound	Transport → Mattermost	Your server is the **receiver**. Subscribes to or polls messages from the configured transport.

A server can have both inbound and outbound connections simultaneously for bidirectional relay. Multiple connections of the same direction are supported for multi-server topologies.

Provider Selection

Each connection uses one of four transport providers: **NATS**, **Azure Queue Storage**, **Azure Blob Storage**, or **Azure Service Bus**. The provider is selected when creating a connection (`provider` field set to `nats`, `azure-queue`, `azure-blob`, or `azure-servicebus`), and the admin console shows only the fields relevant to the chosen provider.

Provider	Transport	File Transfer	Delivery
NATS	Pub/sub messaging via NATS Core	JetStream Object Store	At-most-once (framework retries on hook-return error)
Azure Queue Storage	Polling-based queue (configurable; default 5 s)	Azure Blob Storage (paired container; configurable poll, default 15 s)	At-least-once
Azure Blob Storage	Write-ahead log batched to blobs (configurable flush, default 60 s; configurable inbound poll, default 30 s)	Deferred file blobs alongside batch WAL blobs in the same container	At-least-once
Azure Service Bus	AMQP queue with PeekLock pull (32-message batch, 5 s max wait)	Azure Blob Storage sidecar (independent credentials; configurable poll, default 15 s)	At-least-once with native DLQ on `MaxDeliveryCount` exceedance

Common Connection Fields

Field	JSON Key	Required	Validation	Description
Name	`name`	Yes	Lowercase letters, numbers, and hyphens only. Pattern: `^[a-z0-9]+(-[a-z0-9]+)*$`. Must be unique within its direction (inbound or outbound). The same name may appear in both an inbound and an outbound entry, which is the loopback configuration.	Human-readable identifier. Used in all commands and API calls. Examples: `relay-to-b`, `high-side`, `nats-prod`
Provider	`provider`	Yes	`"nats"`, `"azure-queue"`, or `"azure-blob"`. Empty treated as `"nats"`.	Transport provider for this connection. Determines which provider-specific fields

Field	JSON Key	Required	Validation	Description
				are required. Default: `"nats"`.
File Transfer Enabled	`file_transfer_enabled`	No	Boolean	Enable file attachment relay for this connection. Default: `false`.
File Filter Mode	`file_filter_mode`	No	`""`, `"allow"`, or `"deny"`	Controls whether file extensions are filtered. `allow` = only listed extensions are relayed. `deny` = listed extensions are blocked. Empty = all files pass (default).
File Filter Types	`file_filter_types`	Conditional	Comma-separated extensions. Case-insensitive, leading dot optional.	List of file extensions used by the filter (e.g. `.pdf,.docx,.png`). Required when `file_filter_mode` is `allow` or `deny`.
Message Format	`message_format`	No	`"json"` or `"xml"`. Empty treated as `"json"`.	Wire format for outbound messages. Use `"xml"` when sending through a Cross Domain Solution (CDS) that requires XML. Inbound messages are auto-detected regardless of this setting. Only applies to outbound connections. Default: `"json"`.

NATS Provider Fields

These fields appear when the provider is set to **NATS**. In JSON, they are nested under the `"nats"` key (e.g., `"nats": {"address": "...", "subject": "..."}`).

Field	JSON Key	Required	Validation	Description
Address	`address`	Yes	Non-empty string	NATS server URL. Format: `nats://hostname:port`. Default port is 4222. Examples: `nats://nats.example.com:4222`, `nats://10.0.1.50:4222`
Subject	`subject`	Yes	Must start with `crossguard.` prefix	NATS subject to publish/subscribe on. Both sides MUST use the same subject. Examples: `crossguard.relay.messages`, `crossguard.prod.sync`
TLS Enabled	`tls_enabled`	No	Boolean	Enable TLS encryption. Required for production with TLS-enabled NATS servers.
Auth Type	`auth_type`	No	`none`, `token`, or `credentials`. Empty treated as `none`.	Authentication method for the NATS server.
Token	`token`	Conditional	Required when `auth_type` is `token`	Authentication token for token-based auth.
Username	`username`	Conditional	Required when `auth_type` is `credentials`	Username for credential-based auth.
Password	`password`	Conditional	Required when `auth_type` is `credentials`	Password for credential-based auth.
Client Certificate	`client_cert`	No	If provided, `client_key` must also be provided	Path to client TLS certificate file (requires `tls_enabled`).

Field	JSON Key	Required	Validation	Description
Client Key	`client_key`	No	If provided, `client_cert` must also be provided	Path to client TLS private key file.
CA Certificate	`ca_cert`	No	-	Path to CA certificate for verifying the NATS server's TLS certificate.

Azure Queue Provider Fields

These fields appear when the provider is set to **Azure Queue Storage**. In JSON, they are nested under the `"azure_queue"` key (e.g., `"azure_queue": {"queue_service_url": "...", "account_name": "...", "account_key": "...", "queue_name": "..."}`).

Field	JSON Key	Required	Validation	Description
Queue Service URL	`queue_service_url`	Yes	Non-empty, valid URL	Azure Queue Storage service URL. For production Azure, the standard form is `https://<account>.queue.core.windows.net`. For Azurite (local emulator), use `http://azurite:10001/devstoreaccount1`.
Blob Service URL	`blob_service_url`	Conditional	Required when `file_transfer_enabled` is `true`. Must be a valid URL.	Azure Blob Storage service URL. For production Azure, the standard form is `https://<account>.blob.core.windows.net`.
Account Name	`account_name`	Yes	Non-empty string	Azure Storage account name. Paired with `account_key` to form a shared-key credential.

Field	JSON Key	Required	Validation	Description
Account Key	`account_key`	Yes	Non-empty string	Azure Storage account shared access key (base64). Treat as a secret. Obtain from the Azure Portal under Storage Account > Access Keys.
Queue Name	`queue_name`	Yes	Non-empty string	Name of the Azure Queue. The plugin auto-creates the queue if it does not exist. Both sides must use the same queue name.
Blob Container Name	`blob_container_name`	Conditional	Required when `file_transfer_enabled` is `true`	Name of the Azure Blob container for file transfer. Auto-created if it does not exist. Both sides must use the same container name.

Azure Blob Provider Fields

These fields appear when the provider is set to **Azure Blob Storage** (the batched WAL transport, distinct from the Azure Queue provider above). In JSON, they are nested under the `"azure_blob"` key. Instead of a queue, the outbound side batches messages into a write-ahead log blob and flushes it on an interval; the inbound side polls the container, takes a short-lived lease on each batch blob, processes it, and deletes it. File attachments are stored as deferred blobs alongside the batch blobs in the same container.

Field	JSON Key	Required	Validation	Description
Service URL	`service_url`	Yes	Non-empty, valid URL	Azure Blob Storage service URL. For production Azure, the standard form is `https://<account>.blob.core.windows.net`. For Azurite, use `http://azurite:10000/devstoreaccount1`.

Field	JSON Key	Required	Validation	Description
Account Name	`account_name`	Yes	Non-empty string	Azure Storage account name. Paired with `account_key` to form a shared-key credential.
Account Key	`account_key`	Yes	Non-empty string	Azure Storage account shared access key (base64). Treat as a secret.
Blob Container Name	`blob_container_name`	Yes	Non-empty string	Name of the Azure Blob container holding both batch WAL blobs and (when file transfer is enabled) deferred file blobs. Auto-created if it does not exist. Both sides must use the same container name.
Flush Interval	`flush_interval_seconds`	No	Integer. If set, must be at least `5`.	How often the outbound side seals the current write-ahead log blob and uploads it. Default: `60` seconds. Lower values reduce end-to-end latency at the cost of more blob operations.
Blob Lock Max Age	`blob_lock_max_age_seconds`	No	Integer. If set, must be at least `30`.	Maximum age of a blob lease before the inbound side considers it stale and takes over processing. Default: `300` seconds (5 minutes). Prevents a crashed inbound node from stalling delivery.

Azure Service Bus Provider Fields

These fields appear when the provider is set to **Azure Service Bus**. In JSON, they are nested under the `"azure_servicebus"` key. Service Bus uses an AMQP-based queue for message relay with PeekLock semantics: the plugin completes messages on successful handler invocation and abandons them on handler error (Service Bus then increments delivery count and routes to the dead-letter queue after the queue-configured `MaxDeliveryCount`).

Important: The plugin does *not* auto-create the queue. Pre-create it in the Azure Portal or via ARM/Terraform with your chosen `LockDuration` (default 60 s, maximum 5 min) and

`MaxDeliveryCount` before saving the connection. Lock duration is an entity property; the plugin cannot configure it.

Why the blob fields are named differently from the Azure Queue provider: Service Bus and Azure Storage are separate Azure resources with independent credentials. The Azure Queue provider shares one storage account for both queues and blobs; the Service Bus provider needs a distinct credential pair for its Blob sidecar (which is why you see `blob_account_name` / `blob_account_key` here instead of the unqualified `account_name` / `account_key`).

Field	JSON Key	Required	Validation	Description
Connection String	`connection_string`	Yes	Non-empty string	SAS connection string, e.g. `Endpoint=sb://<namespace>.servicebus.windows.net/;SharedAccessKeyName=<rule>;SharedAccessKey=<key>`. Treat as a secret; never exposed in status responses or logs.
Queue Name	`queue_name`	Yes	Up to 260 characters; letters, digits, periods, hyphens, underscores, and forward slashes.	Name of the pre-created Service Bus queue. Hierarchical paths such as `team1/crossguard-relay` are allowed.
Blob Service URL	`blob_service_url`	Only when `file_transfer_enabled` is true	Valid URL	Azure Blob Storage endpoint for the file sidecar container.
Blob Account Name	`blob_account_name`	Only when `file_transfer_enabled` is true	Non-empty string	Storage account name for the blob sidecar. Independent of the Service Bus namespace.

Field	JSON Key	Required	Validation	Description
Blob Account Key	`blob_account_key`	Only when `file_transfer_enabled` is true	Non-empty string	Shared access key for the blob sidecar storage account. Treat as a secret.
Blob Container Name	`blob_container_name`	Only when `file_transfer_enabled` is true	Non-empty string	Blob container used for file attachment relay. Auto-created on startup if it does not exist.
Max Message Size	`max_message_size_bytes`	No	Integer, 1024 to 104857600. Default `192000`.	Maximum outbound message payload. Oversized messages are rejected at the client boundary. Raise this to 100 MiB only on the Premium tier.
Blob Poll Interval	`blob_poll_interval_seconds`	No	Integer, ≥ 1. Default `15`.	Inbound poll cadence for the blob sidecar when file transfer is enabled.

Dead-Letter Queue (DLQ) behaviour: If handler errors cause redelivery count to exceed the queue's `MaxDeliveryCount`, Azure Service Bus auto-moves the message to the queue's dead-letter sub-queue. The plugin does NOT inspect or process DLQ messages; they stop appearing in receive polls. Use the Azure Portal, `az servicebus queue message` CLI, or Azure monitoring alerts to inspect the DLQ. The plugin logs a WARN with error code `26004` (`ServiceBusRedelivery`) on every redelivery so operators can detect poison-message buildup before DLQ moves happen.

Azure Service Principal Authentication

All three Azure providers (Queue, Blob, Service Bus) support **Azure AD Service Principal authentication** as an alternative to shared keys / SAS connection strings. Service Principal uses the AAD `client_credentials` grant: an application registered in Azure AD authenticates with a tenant ID, a client (application) ID, and a client secret. The plugin acquires bearer tokens from AAD and uses them to call the Azure data plane.

Why this matters: Shared keys grant full account access and cannot be scoped or rotated centrally. Service Principal + Azure RBAC lets you grant a single AAD identity exactly the permissions it needs on a specific queue, container, or namespace, and rotation happens in AAD without touching plugin config.

Phase 1 Deployment Guidance

This release supports the **Client Secret** flow only. Client certificate auth and Managed Identity are tracked as later phases. From a compliance / threat-model perspective:

- **Acceptable for:** development, staging, and production environments paired with a strict secret-rotation policy (90-day maximum). Federal customers using Azure Government with rotation tooling can deploy Phase 1 in production.
- **Not the recommended posture for:** high-side ATO environments where shared-secret material at rest is a finding. For those environments, wait for Phase 2 (certificate auth via Azure Key Vault + CSI driver) or use the Mattermost env-var substitution mechanism described below to keep the secret out of plugin config entirely.

Known Exposure: Secret in Plugin Config (Phase 1B Follow-Up)

The Mattermost `GET /api/v4/config` endpoint returns plugin custom-settings as cleartext to any System Admin. This affects **all** Crossguard secret fields (`account_key`, `connection_string`, `blob_account_key`, `client_secret`): an admin opening the System Console will see the secret value in their browser's network tab and any `mmctl config show` output will include it. A structural fix that replaces the cleartext with a reference is tracked as Phase 1B.

Immediate Federal mitigation: inject the entire connections JSON via Mattermost's plugin-settings env-var substitution. Setting `MM_PLUGINSETTINGS_PLUGINS_CROSSGUARD_OUTBOUNDCONNECTIONS` (and the inbound counterpart) overrides whatever is stored in plugin config at load time; the value can come from a Kubernetes Secret, a CSI volume + init container, systemd `LoadCredential=`, or HashiCorp Vault. The credential then lives only in the Mattermost process environment, not in `config.json` on disk, not in the configuration database, and not in `GET /api/v4/config` responses. This is the same mechanism Mattermost uses for every other plugin secret.

Common Service Principal Fields

The following fields are shared across all three Azure providers when `auth_mode` is `service-principal`:

Field	Required	Type	Description
`auth_mode`	No	String enum. Queue/Blob: `"shared-key"` (default) or `"service-principal"`. Service Bus: `"connection-string"` (default) or `"service-principal"`.	Selects the credential type. Empty defaults to the legacy mode for backward compatibility (existing configs keep working unchanged).
`azure_cloud`	No	String enum: `"public"` (default), `"usgov"`, `"china"`.	Selects the Azure cloud environment. Passed to the AAD authority host for credential acquisition. For Queue and Blob, also passed to the data-plane SDK client options. For Service Bus, data-plane routing comes from the FQDN namespace, so only the credential receives the cloud option.
`tenant_id`	Yes (SP)	String. GUID or domain form (e.g., `contoso.onmicrosoft.com`).	Azure AD directory (tenant) ID.
`client_id`	Yes (SP)	String (GUID).	Azure AD application (client) ID for the Service Principal.
`client_secret`	Yes (SP)	String (secret).	Inline client secret value. Required in SP mode. To keep the secret out of plugin config on disk, inject the whole connections JSON via the `MM_PLUGINSETTINGS_PLUGINS_CROSSGUARD_OUTBOUNDCONNECTIONS` / `INBOUNDCONNECTIONS` env var (see Phase 1B mitigation above).

Azure Service Bus – Additional SP Field

Field	Required	Type	Description
`service_bus_namespace`	Yes (SP)	String (FQDN).	Fully-qualified Service Bus namespace, e.g., `myns.servicebus.windows.net` for Public, `myns.servicebus.usgovcloudapi.net` for US Government. The SAS connection string used to

Field	Required	Type	Description
			encode this for the legacy mode; in SP mode it must be supplied separately. If you accidentally include the `sb://` scheme, it is stripped.

Minimum Required Azure RBAC Roles

Grant the Service Principal these data-plane roles at the resource scope. The plugin **skips auto-creation** of queues/containers in SP mode — you must pre-provision them via the Azure Portal, ARM template, or `az` CLI. This means the operator does not need management-plane (Owner / Contributor) roles, which is the correct least-privilege posture.

- **Azure Queue provider:** `Storage Queue Data Contributor` on the Queue Storage account. If file transfer is enabled, also grant `Storage Blob Data Contributor` on the same account.
- **Azure Blob provider:** `Storage Blob Data Contributor` on the Blob Storage account.
- **Azure Service Bus provider:** `Azure Service Bus Data Sender` AND `Azure Service Bus Data Receiver` on the queue (or on the namespace). If file transfer is enabled and the blob sidecar inherits the parent SP credential (the default in SP mode), the same SP also needs `Storage Blob Data Contributor` on the blob storage account.

Owner-tier roles are NOT required. If you see startup errors about `AuthorizationPermissionMismatch` on a Create call, double-check that the queue/container has been pre-provisioned and that the SP has the correct data-plane role.

Service Bus Blob Sidecar Credential Inheritance (SP Mode)

When Azure Service Bus is in SP mode the blob sidecar **inherits the parent SP credential**: do NOT set `blob_account_name` or `blob_account_key`, validation rejects the save if you do. This empty-fields rule applies unconditionally in SP mode, even when `file_transfer_enabled` is false, so a stored shared-key cannot linger across a file-transfer toggle. When file transfer is enabled, provide `blob_service_url` and `blob_container_name`. The same Service Principal must have the appropriate roles on both the Service Bus namespace and the blob storage account. Mixed-credential deployments (one SP for AMQP, a different SP or shared key for blob) are deferred to Phase 2.

Azure Government (Sovereign Cloud) Setup

To configure a connection for Azure US Government:

1. Set `azure_cloud` to `usgov`.
2. Use the US Gov endpoints in `queue_service_url`, `blob_service_url`, `service_url`, or `service_bus_namespace`: `*.queue.core.usgovcloudapi.net`, `*.blob.core.usgovcloudapi.net`, `*.servicebus.usgovcloudapi.net`.
3. Register the Service Principal in the Azure US Government AAD tenant (not the commercial tenant). Grant the data-plane RBAC roles described above.

The plugin passes the cloud configuration to `azidentity` so token acquisition routes to `login.microsoftonline.us` automatically; you do not need to override an authority host.

Save-Time Credential Probe

When you save a plugin configuration that includes one or more SP connections, the plugin acquires a token from AAD for each SP connection before the save commits. Bad tenant/client/secret values fail the save with a sanitized error message instead of producing repeated runtime failures during the relay loop. The probe has a 10-second timeout. Errors are scrubbed of any credential material before being logged or surfaced to the admin UI.

Secret Hydration in the Admin Console

When you open an existing connection in the System Console, password fields (including `client_secret`, `account_key`, `connection_string`, `blob_account_key`) are hydrated with a distinctive sentinel token (`__CROSSGUARD_SECRET_UNCHANGED__`) instead of the stored cleartext. The token is rendered as bullets in the password input and never round-trips to a browser as a real secret. If you save the form without editing a password field, the sentinel is sent back to the server and the stored value is preserved.

- **To rotate a secret:** paste the new value into the field and save.
- **To leave a secret unchanged:** do nothing — do not delete the field contents.
- **To clear a stored secret** (for example, when migrating from shared-key to service-principal): clear the field and save. An empty inbound value is honored as an explicit clear.

Validation Rules Summary

1. Names must be unique within each direction (inbound or outbound). The same bare name may appear in both lists, which is the loopback configuration where one server is both publishing and subscribing on the same transport
2. Names must be lowercase alphanumeric with hyphens (no spaces, underscores, uppercase)

3. Subject must always start with `crossguard.`
4. Both sides of a relay must use the same NATS subject
5. Auth credentials must be complete (both username AND password for credentials auth)
6. TLS cert and key must be provided as a pair (enforced at config save time)
7. Azure Queue connections require `queue_service_url` and `queue_name`; shared-key mode additionally requires `account_name` and `account_key`
8. Azure Queue connections require `blob_service_url` and `blob_container_name` when file transfer is enabled
9. Azure Blob connections require `service_url` and `blob_container_name`; shared-key mode additionally requires `account_name` and `account_key`
10. Azure Blob `flush_interval_seconds` must be at least 5 when set; `blob_lock_max_age_seconds` must be at least 30 when set
11. Azure Service Bus connections require `queue_name`; connection-string mode additionally requires `connection_string`; service-principal mode additionally requires `service_bus_namespace`
12. Azure Service Bus `queue_name` is capped at 260 characters and allows only `A-Z a-z 0-9 . _ - /`
13. Azure Service Bus connections with file transfer enabled also require `blob_service_url` and `blob_container_name`; in connection-string mode also `blob_account_name` and `blob_account_key`. In service-principal mode `blob_account_name` and `blob_account_key` must be **empty** (sidecar inherits parent SP); this empty-fields rule applies unconditionally in SP mode, including when file transfer is disabled, so stale shared-key material cannot survive a file-transfer toggle
14. When `auth_mode` is `service-principal`: `tenant_id` (GUID or FQDN), `client_id`, and `client_secret` are required; the corresponding legacy field (`account_key` / `connection_string`) must be empty
15. When `auth_mode` is set, `azure_cloud` must be one of `public`, `usgov`, or `china` when present; empty defaults to `public`

Relay Settings

Restrict to System Admins

Setting	`RestrictToSystemAdmins`
Type	Boolean
Default	`false`

When enabled: Only System Admins can run `init-team`, `teardown-team`, `init-channel`, `teardown-channel`, `reset-prompt`, `reset-channel-prompt`, and `rewrite-team`. Team Admins and Channel Admins are denied access.

When disabled: Team Admins can manage team connections, Channel Admins can manage channel connections (the default permission hierarchy).

> **Tip**
> Enable this setting in high-security environments where only centralized admins should control cross-domain relay. Pair it with `AllowTeamAdminRequests` below to let Team Admins request connections without granting them direct control.

Allow Team Admin Requests

Setting	`AllowTeamAdminRequests`
Type	Boolean
Default	`false`
Requires	`RestrictToSystemAdmins` must also be enabled

When enabled (and `RestrictToSystemAdmins` is also enabled): Team Admins can submit connection link requests via `/crossguard init-team` or the REST API. The request is sent as a direct message to every System Admin with interactive **Approve** and **Deny** buttons. Team Admins can also unlink connections directly with `/crossguard teardown-team` (no approval needed for unlinking).

When disabled: Team Admins are fully blocked from connection management when `RestrictToSystemAdmins` is enabled. This has no effect when `RestrictToSystemAdmins` is disabled, because Team Admins already have direct access.

> **Request workflow**
> When a Team Admin submits a link request, all System Admins receive a DM from the Cross Guard bot with the requester's name, team, and connection. The DM contains **Approve** and **Deny** buttons. Approving executes the link immediately. Denying opens an optional reason dialog. The requester is notified of the outcome via DM. If the connection is unlinked while a request is pending, the request is automatically cancelled.

Allow Channel Admin Requests

Setting	`AllowChannelAdminRequests`
Type	Boolean
Default	`false`
Requires	`RestrictToSystemAdmins` must also be enabled

When enabled (and `RestrictToSystemAdmins` is also enabled): Channel Admins can submit channel connection link requests via `/crossguard init-channel` or the REST API. The request is sent as a direct message to every Team Admin with interactive **Approve** and **Deny** buttons. Channel Admins can also unlink channel connections directly with `/crossguard teardown-channel` (no approval needed for unlinking).

When disabled: Channel Admins are fully blocked from channel connection management when `RestrictToSystemAdmins` is enabled. This has no effect when `RestrictToSystemAdmins` is disabled, because Channel Admins already have direct access.

> **Request workflow**
> When a Channel Admin submits a channel link request, all Team Admins for that team receive a DM from the Cross Guard bot with the requester's name, team, channel, and connection. The DM contains **Approve** and **Deny** buttons. Approving executes the channel link immediately. Denying opens an optional reason dialog. The requester is notified of the outcome via DM. If the connection is unlinked while a request is pending, the request is automatically cancelled.

File Transfer Settings

File transfer is configured per connection, not globally. Each connection can independently enable or disable file attachment relay and apply extension filters.

Requirements

NATS Provider

- **JetStream-enabled NATS server:** File transfer uses NATS JetStream Object Store. Core NATS alone is not sufficient. The NATS server must have JetStream enabled.
- **Object Store bucket:** The plugin auto-creates a `crossguard-files` bucket with a 1-hour TTL on first use. No manual NATS configuration is needed.
- **Max file size:** Governed by the Mattermost server's `FileSettings.MaxFileSize` configuration (default 100 MB). Files exceeding this limit are silently skipped with a log warning.

Azure Provider

- **Blob container:** File transfer uses Azure Blob Storage. The plugin auto-creates the container specified in `blob_container_name` if it does not exist.
- **Blob metadata:** Headers (post ID, connection name, filename) are stored as Base64-encoded JSON in the blob's `crossguard_headers` metadata key.
- **Polling interval:** The inbound blob watcher polls every 15 seconds. Files are deleted after successful processing.
- **Max file size:** Same as NATS: governed by the Mattermost server's `FileSettings.MaxFileSize` configuration.

Filter Examples

Goal	file_filter_mode	file_filter_types
Allow all files (default)	`""`	`""`
Allow only PDFs and images	`"allow"`	`".pdf,.png,.jpg,.gif"`
Block executables	`"deny"`	`".exe,.bat,.sh,.cmd"`

> **Note**

> For bidirectional file relay, both the outbound and inbound connections must have `file_transfer_enabled` set to `true`. File filters are evaluated independently on each side, so the outbound connection can filter what it sends and the inbound connection can filter what it accepts.

Testing Connections

The admin console UI includes a "Test Connection" button for each configured connection. The handler dispatches by `provider` to a per-provider test implementation; each issues a temporary client and does not affect the running plugin's connections.

- **API endpoint:** `POST /plugins/crossguard/api/v1/test-connection` (System Admin only). Accepts a full `ConnectionConfig` as the request body and an optional `direction` query parameter.
- **NATS**: outbound test publishes a test envelope (with the current Epoch and a generated TestID) on the configured subject and confirms flush; inbound test subscribes to the subject and confirms the subscription is active. The `direction` query parameter selects which.
- **Azure Queue Storage**: validates the configured credential (shared key or Service Principal) by listing the target queue and, on outbound, enqueueing and deleting a test message.
- **Azure Blob Storage**: validates the credential against the target container and verifies the plugin can read and write a probe blob.
- **Azure Service Bus**: opens an AMQP connection to the namespace, validates the credential, and on outbound sends a test envelope to the configured queue. SAS material and SP errors are sanitized before being surfaced to the admin UI.
- Service Principal credentials trigger a save-time AAD probe in addition to the data-plane test. AAD errors are sanitized before being returned to the admin UI.

Example Configurations

Dual-Server Setup (Server A sends to Server B)

Server A (sender)

```
Outbound Connections:
[{
  "name": "relay-to-b",
  "provider": "nats",
  "nats": {
    "address": "nats://nats.example.com:4222",
    "subject": "crossguard.relay.messages",
    "auth_type": "token",
    "token": "shared-secret-token"
  },
  "file_transfer_enabled": true,
  "file_filter_mode": "allow",
  "file_filter_types": ".pdf,.docx,.png"
}]

Inbound Connections: []
```

Server B (receiver)

```
Inbound Connections:
[{
  "name": "relay-from-a",
  "provider": "nats",
  "nats": {
    "address": "nats://nats.example.com:4222",
    "subject": "crossguard.relay.messages",
    "auth_type": "token",
    "token": "shared-secret-token"
  },
  "file_transfer_enabled": true
}]

Outbound Connections: []
```

Bidirectional Setup (Both servers send and receive)

Server A

```
Outbound: [{"name": "to-b", "provider": "nats", "nats": {"address": "nats://nats:42
22", "subject": "crossguard.a-to-b"}}]
```

```
Inbound:  [{"name": "from-b", "provider": "nats", "nats": {"address": "nats://nats:
4222", "subject": "crossguard.b-to-a"}}]
```

Server B

```
Outbound: [{"name": "to-a", "provider": "nats", "nats": {"address": "nats://nats:42
22", "subject": "crossguard.b-to-a"}}]
Inbound:  [{"name": "from-a", "provider": "nats", "nats": {"address": "nats://nats:
4222", "subject": "crossguard.a-to-b"}}]
```

> **Note**
> Each direction uses a different subject to avoid echo loops. Server A publishes on `crossguard.a-to-b` and Server B subscribes to it, and vice versa.

Docker Dev Environment

- **Server A:** `http://low.test:8075` (admin/password, usera/password, Team: Test A)
- **Server B:** `http://high.test:8076` (admin/password, userb/password, Team: Test B)
- **NATS:** `nats://nats:4222` (from plugin's perspective inside Docker network)
- **Setup:** `make docker-setup`, **Deploy:** `make deploy`

Azure Queue Storage Setup

Server A (sender via Azure Queue)

```
Outbound Connections:
[{
  "name": "azure-to-b",
  "provider": "azure-queue",
  "azure_queue": {
    "queue_service_url": "https://mystorageaccount.queue.core.windows.net",
    "blob_service_url": "https://mystorageaccount.blob.core.windows.net",
    "account_name": "mystorageaccount",
    "account_key": "base64-account-key",
    "queue_name": "crossguard-relay",
    "blob_container_name": "crossguard-files"
  },
```

```
  "file_transfer_enabled": true
}]
```

Server B (receiver via Azure Queue)

```
Inbound Connections:
[{
  "name": "azure-from-a",
  "provider": "azure-queue",
  "azure_queue": {
    "queue_service_url": "https://mystorageaccount.queue.core.windows.net",
    "blob_service_url": "https://mystorageaccount.blob.core.windows.net",
    "account_name": "mystorageaccount",
    "account_key": "base64-account-key",
    "queue_name": "crossguard-relay",
    "blob_container_name": "crossguard-files"
  },
  "file_transfer_enabled": true
}]
```

Azure Blob variant (batched WAL transport)

```
Outbound Connections:
[{
  "name": "azure-blob-to-b",
  "provider": "azure-blob",
  "azure_blob": {
    "service_url": "https://mystorageaccount.blob.core.windows.net",
    "account_name": "mystorageaccount",
    "account_key": "base64-account-key",
    "blob_container_name": "crossguard-wal",
    "flush_interval_seconds": 60,
    "blob_lock_max_age_seconds": 300
  },
  "file_transfer_enabled": true
}]
```

Azure Service Bus variant (AMQP with PeekLock and DLQ)

```
Outbound Connections:
[{
```

```
  "name": "azure-servicebus-to-b",
  "provider": "azure-servicebus",
  "azure_servicebus": {
    "connection_string": "Endpoint=sb://myns.servicebus.windows.net/;SharedAccessKe
yName=RootManageSharedAccessKey;SharedAccessKey=...",
    "queue_name": "crossguard-relay",
    "blob_service_url": "https://mystorageaccount.blob.core.windows.net",
    "blob_account_name": "mystorageaccount",
    "blob_account_key": "base64-account-key",
    "blob_container_name": "crossguard-files"
  },
  "file_transfer_enabled": true
}]
```

Service Bus with Service Principal authentication

```
Outbound Connections:
[{
  "name": "azure-servicebus-to-b",
  "provider": "azure-servicebus",
  "azure_servicebus": {
    "auth_mode": "service-principal",
    "azure_cloud": "public",
    "service_bus_namespace": "myns.servicebus.windows.net",
    "tenant_id": "00000000-0000-0000-0000-000000000000",
    "client_id": "11111111-1111-1111-1111-111111111111",
    "client_secret": "...",
    "queue_name": "crossguard-relay",
    "blob_service_url": "https://mystorageaccount.blob.core.windows.net",
    "blob_container_name": "crossguard-files"
  },
  "file_transfer_enabled": true
}]
```

> **Note**
> For NATS, Azure Queue, and Azure Blob, both sides typically share the same provider configuration. The outbound side publishes / enqueues / appends; the inbound side subscribes / polls / leases. For Azure Service Bus, the queue is pre-created (the plugin does not auto-create it) with the desired `LockDuration` and `MaxDeliveryCount`; the blob sidecar is a separate storage account (in connection-string mode) or inherits the parent Service Principal (in SP mode). Service Principal mode is the recommended posture for production Federal deployments.

Troubleshooting

Problem	Solution
"No connections configured"	Add connections in System Console > Plugins > Cross Guard first.
NATS connection test fails	Verify NATS server is reachable, check address/port, verify auth credentials match.
Messages not relaying	Run `/crossguard status`. Verify both team AND channel are initialized. Check that transport identifiers match on both sides (NATS subject for NATS connections, queue name for Azure connections).
"Connection not found"	Connection name is case-sensitive and uses `direction:name` format (e.g., `outbound:relay-to-b`).
Orphaned connections	If a connection name is changed in config, existing team/channel links reference the old name. Teardown and re-init with the new name.
Inbound messages blocked	Check if an inbound prompt was blocked. Run `/crossguard reset-prompt <name>` to clear it and re-evaluate.
Wrong team receives messages	Check team name rewrite settings with `/crossguard status`. Remote team names must match or be rewritten.
Files not transferring	Verify `file_transfer_enabled` is `true` on both outbound and inbound connections. For NATS, ensure the server has JetStream enabled (not just core NATS). For Azure, verify `blob_container_name` is set on both outbound and inbound connections.
File blocked by filter	Check `file_filter_mode` and `file_filter_types` on the connection. An `allow` filter rejects extensions not in the list; a `deny` filter rejects extensions in the list. Filtering is applied on both outbound and inbound sides.
File appears after text message	Normal behavior. Files ride a parallel file-upload channel (NATS Object Store, Azure Blob container, or Azure Service Bus blob sidecar) and arrive asynchronously. The shared-channels framework re-resolves the post to attach to once both arrive.

Problem	Solution
Large files silently dropped	Files larger than the server's `MaxFileSize` setting (default 100 MB) are skipped with a log warning. Check `FileSettings.MaxFileSize` in the Mattermost server configuration.
Azure connection test fails (shared-key / SAS)	Verify the connection string is correct. Check that the storage account or Service Bus namespace is accessible from the server. Ensure the account key or SAS has not been rotated.
Azure connection test fails (Service Principal)	The save-time AAD probe verifies that `tenant_id`, `client_id`, and `client_secret` can authenticate against the configured `azure_cloud`. Errors are sanitized but typically point at a wrong tenant ID, an expired secret, or missing data-plane RBAC at the configured scope (Storage Queue Data Contributor / Storage Blob Data Contributor / Azure Service Bus Data Sender + Receiver).
Azure Queue messages delayed	Azure Queue Storage uses polling. Default cadence is 5 s for messages, 15 s for files; both are configurable via `poll_interval_seconds` and `blob_poll_interval_seconds`. For lower latency, use NATS or Azure Service Bus.
Azure Queue message too large	Azure Queue Storage has a 48,000-byte message size limit (before Base64 expansion fits the 64 KB Azure Queue wire limit). Messages exceeding this limit are rejected at the client boundary. Switch to Azure Blob Storage (batched WAL, no per-message cap) or Azure Service Bus (configurable up to 100 MiB on Premium) for larger payloads.
Duplicate messages with Azure Queue or Blob	Both provide at-least-once delivery. The inbound sequencer keys on `(Epoch, ConnName, ChannelId, Sequence)` and drops duplicates structurally; combined with the single-active-receiver lease (only one cluster node subscribes at a time), duplicates do not reach the framework. If you see suspected duplicates, check the `InboundSeq*` audit codes (15300-range) for the actual delivery pattern.
Azure blob files not appearing	Verify `blob_container_name` is set on both sides. Check that `file_transfer_enabled` is `true` on both outbound and inbound connections. The blob watcher polls at `blob_poll_interval_seconds` cadence (default 15 s for Azure Queue and Service Bus; `batch_poll_interval_seconds` default 30 s for Azure Blob).

Problem	Solution
Azure Service Bus: DeliveryCount warnings	Service Bus PeekLock increments `DeliveryCount` on every Abandon. Error code `26004` logs a WARN when a message arrives with `DeliveryCount > 1`, surfacing poison-message buildup before the broker auto-moves to DLQ. Inspect the DLQ via `az servicebus queue message` or the Azure portal; the plugin does not consume the DLQ.
Azure Service Bus: empty-body messages	The plugin Completes empty-body messages to avoid infinite redelivery loops (error code `26005`). This is normal during pre-existing-queue cleanup; if it persists, investigate the source side.
Azure Service Bus: queue not found	The plugin does NOT auto-create Service Bus queues. Pre-create the queue in the Azure portal (or via ARM / Terraform) with the desired `LockDuration` and `MaxDeliveryCount` before configuring the connection.
Azure Service Bus SP: blob sidecar credentials	In `auth_mode: "service-principal"` mode, the blob sidecar inherits the parent SP credential; `blob_account_name` and `blob_account_key` must be empty. Mixed-mode sidecars (SP for messages, shared-key for files) are not supported. Grant the SP *Storage Blob Data Contributor* on the sidecar container.
Inbound stalls in a clustered deployment	Only one cluster node holds the active inbound subscription at a time (KV TTL lease, 45 s TTL with 15 s renewal). If that node loses KV access, failover takes up to one TTL. Watch the `InboundActive*` audit codes (15200-range) for the election story.
Inbound sequence gaps in operator logs	The inbound sequencer audits ranges whose missing predecessors never arrived after the 1 s gap deadline, via the `InboundSeq*` codes (15300-range). Investigate alongside the source-side `OutboundSyncMsg*` codes (10100-range) to see whether the gap was created by the sender, the transport, or the receiver.

REST API Reference

All Cross Guard REST endpoints with request and response examples

Overview

Base Path	`/plugins/crossguard/api/v1/`
Authentication	Most endpoints require the `Mattermost-User-Id` header (set automatically by Mattermost for authenticated requests). The interactive-dialog handler `POST /api/v1/dialog/select-connection` instead reads the user ID from the `user_id` field of the `SubmitDialogRequest` body that Mattermost POSTs when a dialog is submitted, so the header is not required there.
Request Limit	5 MB max request body
Response Format	JSON
Error Format	`{"error": "message"}` with appropriate HTTP status code

Connection Testing

POST `/api/v1/test-connection` SYSTEM ADMIN

Test a connection configuration before saving. Supports NATS, Azure Queue, Azure Blob, and Azure Service Bus providers.

Query Parameters

`direction`	Optional. `inbound` or `outbound` (defaults to `outbound`). NATS-specific; ignored for Azure Queue, Azure Blob, and Azure Service Bus connections.

Request Body (NATS)

```
{
  "name": "relay-to-b",
```

```
  "provider": "nats",
  "nats": {
    "address": "nats://nats.example.com:4222",
    "subject": "crossguard.relay.messages",
    "tls_enabled": false,
    "auth_type": "token",
    "token": "my-secret-token"
  }
}
```

Request Body (Azure Queue)

```
{
  "name": "azure-relay",
  "provider": "azure-queue",
  "azure_queue": {
    "queue_service_url": "https://mystorageaccount.queue.core.windows.net",
    "account_name": "mystorageaccount",
    "account_key": "base64-account-key",
    "queue_name": "crossguard-relay"
  }
}
```

Request Body (Azure Blob)

```
{
  "name": "azure-blob-relay",
  "provider": "azure-blob",
  "azure_blob": {
    "service_url": "https://mystorageaccount.blob.core.windows.net",
    "account_name": "mystorageaccount",
    "account_key": "base64-account-key",
    "blob_container_name": "crossguard-wal"
  }
}
```

Request Body (Azure Service Bus)

```
{
  "name": "azure-servicebus-relay",
  "provider": "azure-servicebus",
```

```
  "azure_servicebus": {
    "connection_string": "Endpoint=sb://myns.servicebus.windows.net/;SharedAc
cessKeyName=RootManageSharedAccessKey;SharedAccessKey=...",
    "queue_name": "crossguard-relay"
  }
}
```

Validation

NATS

- `nats.address` required
- `nats.subject` required and must start with `crossguard.`
- `nats.auth_type` must be `none`, `token`, `credentials`, or empty
- `nats.token` required if auth_type is `token`
- `nats.username` + `nats.password` required if auth_type is `credentials`

Azure Queue

- `azure_queue.queue_service_url` required
- `azure_queue.queue_name` required
- `azure_queue.account_name` and `azure_queue.account_key` required when `azure_queue.auth_mode` is empty or `shared-key`; **must be empty** when `auth_mode` is `service-principal`
- When `file_transfer_enabled` is true, `blob_service_url` and `blob_container_name` are also required

Azure Blob

- `azure_blob.service_url` required
- `azure_blob.blob_container_name` required
- `azure_blob.account_name` and `azure_blob.account_key` required when `azure_blob.auth_mode` is empty or `shared-key`; **must be empty** when `auth_mode` is `service-principal`

Azure Service Bus

- `azure_servicebus.queue_name` required; at most 260 characters; allowed characters: letters, digits, periods, hyphens, underscores, and forward slashes
- `azure_servicebus.connection_string` required when `auth_mode` is empty or `connection-string`; **must be empty** when `auth_mode` is `service-principal`

- `azure_servicebus.service_bus_namespace` required when `auth_mode` is `service-principal` (FQDN-shaped, e.g., `myns.servicebus.windows.net`; the `sb://` scheme is stripped if present)
- When `file_transfer_enabled` is true, `blob_service_url` and `blob_container_name` are required. In `connection-string` mode, `blob_account_name` and `blob_account_key` are also required; in `service-principal` mode they **must be empty** (the blob sidecar inherits the parent SP credential)
- `azure_servicebus.max_message_size_bytes` (optional) must be between 1024 and 104857600 when set
- `azure_servicebus.blob_poll_interval_seconds` (optional) must be at least 1 when set

Service Principal Auth (all three Azure providers)

When the per-provider `auth_mode` is `service-principal`, the request body must additionally include the shared SP fields (`tenant_id`, `client_id`, `client_secret`, and optionally `azure_cloud`). See the Azure Service Principal Authentication section in the admin guide for the full field reference and minimum RBAC roles. A 400 response is returned when SP validation fails (bad tenant format, missing client_id, missing client_secret, or cross-mode field leakage).

Success Response (outbound) - 200

```
{"status": "ok", "message": "Test message sent with ID: abc123", "id": "abc123"}
```

Success Response (inbound) - 200

```
{"status": "ok", "message": "Connected and subscribed to subject successfully"}
```

Success Response (Azure Queue) - 200

```
{"status": "ok", "message": "Azure Queue Storage connection test successful"}
```

Success Response (Azure Blob) - 200

```
{"status": "ok", "message": "Azure Blob Storage connection test successful"}
```

Success Response (Azure Service Bus) - 200

```
{"status": "ok", "message": "Azure Service Bus connection test successful"}
```

Error Responses

Status	Meaning
`400`	Validation error
`401`	Not authenticated
`403`	Not system admin
`502`	Transport connection failed (NATS unreachable, or Azure credentials/service URL invalid)

Team Management

POST **`/api/v1/teams/{team_id}/init`** TEAM ADMIN+

Link a connection to a team (API equivalent of `/crossguard init-team`).

Path Parameters

`team_id`	26-character Mattermost team ID

Query Parameters

`connection_name`	Optional. Connection in `direction:name` format (e.g., `outbound:relay-to-b`).

Success Response - 200

```
{
  "status": "ok",
  "team_id": "abc123...",
  "team_name": "test-a",
  "connection_name": "outbound:relay-to-b"
}
```

Request Mode Response - 200

When the caller is a Team Admin and both `RestrictToSystemAdmins` and `AllowTeamAdminRequests` are enabled, the endpoint submits a link request instead of linking directly:

```
{
  "status": "request_submitted",
  "message": "Your request to link connection `outbound:relay-to-b` ...",
  "team_id": "abc123...",
  "connection_name": "outbound:relay-to-b"
}
```

Error Responses

Status	Meaning
`400`	Invalid team_id, no connections configured, or connection not found (includes `connections` array of available names)
`403`	Insufficient permissions
`404`	Team not found
`409`	A request is already pending for this connection (request mode only)
`500`	Internal error

POST /api/v1/teams/{team_id}/teardown TEAM ADMIN+

Unlink a connection from a team. If a pending connection link request exists for the same team and connection, it is automatically cancelled.

Path Parameters

`team_id`	26-character Mattermost team ID

Query Parameters

`connection_name`	Optional. Connection to unlink.

Success Response - 200

```
{
  "status": "ok",
  "team_id": "abc123...",
  "team_name": "test-a",
  "connection_name": "outbound:relay-to-b"
}
```

POST /api/v1/channels/{channel_id}/init CHANNEL ADMIN+

Link a connection to a channel (team auto-inits if needed).

Path Parameters

`channel_id`	26-character Mattermost channel ID

Query Parameters

`connection_name`	Optional. Connection in `direction:name` format.

Success Response - 200

```
{
  "status": "ok",
  "channel_id": "xyz789...",
  "channel_name": "town-square",
  "connection_name": "outbound:relay-to-b"
}
```

Channel Request Mode Response - 200

When `RestrictToSystemAdmins` and `AllowChannelAdminRequests` are both enabled and the caller is a Channel Admin (not a Team Admin or System Admin), the endpoint submits a channel connection request instead of linking directly:

```
{
  "status": "request_submitted",
  "message": "Your request to link connection ... has been submitted for team admin approval.",
  "channel_id": "xyz789...",
  "connection_name": "outbound:relay-to-b"
}
```

Error Responses

Status	Meaning
`400`	Invalid input or no connections
`403`	Insufficient permissions
`404`	Channel not found
`409`	A channel request is already pending for this connection
`500`	Internal error

POST **/api/v1/channels/{channel_id}/teardown** CHANNEL ADMIN+

Unlink a connection from a channel. If a pending channel connection request exists for the same connection, it is automatically cancelled.

Path Parameters

`channel_id`	26-character Mattermost channel ID

Query Parameters

`connection_name`	Optional. Connection to unlink.

Success Response - 200

```
{
  "status": "ok",
  "channel_id": "xyz789...",
  "channel_name": "town-square",
  "connection_name": "outbound:relay-to-b"
}
```

Status

GET `/api/v1/status` SYSTEM ADMIN

Global status of all initialized teams and transport connections.

Response - 200

```
{
  "teams": [
    {
      "team_id": "abc123...",
      "team_name": "test-a",
      "display_name": "Test A",
      "linked_connections": [
```

```
        {"direction": "outbound", "connection": "relay-to-b"}
      ]
    }
  ],
  "connections": [
    {
      "name": "relay-to-b",
      "direction": "outbound",
      "provider": "nats",
      "address": "nats://nats:4222",
      "auth_type": "token",
      "subject": "crossguard.relay.messages",
      "file_transfer_enabled": true,
      "file_filter_mode": "allow",
      "file_filter_types": ".pdf,.docx",
      "message_format": "json"
    },
    {
      "name": "azure-relay",
      "direction": "outbound",
      "provider": "azure-queue",
      "queue_name": "crossguard-relay",
      "file_transfer_enabled": false,
      "message_format": "json"
    },
    {
      "name": "azure-blob-relay",
      "direction": "outbound",
      "provider": "azure-blob",
      "blob_container_name": "crossguard-wal",
      "file_transfer_enabled": false,
      "message_format": "json"
    },
    {
      "name": "azure-servicebus-relay",
      "direction": "outbound",
      "provider": "azure-servicebus",
      "queue_name": "crossguard-relay",
      "file_transfer_enabled": false,
      "message_format": "json"
    }
  ]
}
```

> **Note**
> Connection details are redacted. Tokens, passwords, and TLS cert paths are not included.

GET /api/v1/teams/{team_id}/status TEAM MEMBER

Initialization and connection status for a specific team.

Response - 200


```
{
  "team_id": "abc123...",
  "team_name": "test-a",
  "team_display_name": "Test A",
  "initialized": true,
  "request_mode": false,
  "linked_connections": [
    {"direction": "outbound", "connection": "relay-to-b"}
  ],
  "connections": [
    {
      "name": "relay-to-b",
      "direction": "outbound",
      "linked": true,
      "orphaned": false,
      "request_pending": false,
      "file_transfer_enabled": true,
      "file_filter_mode": "allow",
      "file_filter_types": ".pdf,.docx",
      "message_format": "json"
    }
  ]
}
```


Response Fields

orphaned	`true` when the connection is linked to the team but no longer exists in the plugin configuration.

`remote_team_name`	Set when a team name rewrite is configured for this inbound connection.
`request_mode`	`true` when the calling user is a Team Admin and the plugin is in request mode (`RestrictToSystemAdmins` + `AllowTeamAdminRequests` both enabled). Omitted for System Admins.
`request_pending`	`true` when a link request is pending for this unlinked connection. Only populated when the plugin is in request mode.

GET /api/v1/channels/{channel_id}/status CHANNEL MEMBER

Connection status for a specific channel.

Response - 200

```
{
  "channel_id": "xyz789...",
  "channel_name": "town-square",
  "channel_display_name": "Town Square",
  "team_name": "Test A",
  "channel_request_mode": true,
  "team_connections": [
    {
      "name": "relay-to-b",
      "direction": "outbound",
      "linked": true,
      "orphaned": false,
      "request_pending": false,
      "file_transfer_enabled": true,
      "file_filter_mode": "allow",
      "file_filter_types": ".pdf,.docx",
      "message_format": "json"
    }
  ]
}
```

Channel request mode fields

`channel_request_mode` (top-level) is `true` when the caller is a Channel Admin operating in channel request mode (not a Team Admin or System Admin). `request_pending` (per connection) is `true` when an unresolved channel link request exists for that connection. Both fields are omitted when not applicable.

Note
Returns an error for DM and group message channels: "Cross Guard is not available for direct or group messages"

`GET` `/api/v1/channels/connections` AUTHENTICATED

Bulk query which channels have active Cross Guard connections.

Query Parameters

`ids`	Required. Comma-separated channel IDs (max 2048 IDs).

Response - 200

```
{
  "channel_id_1": "outbound:relay-to-b,inbound:relay-from-a",
  "channel_id_2": "outbound:relay-to-b"
}
```

- Only returns channels the user is a member of
- Invalid IDs and channels without connections are omitted
- Used internally by the webapp to show channel indicators

Team Name Rewriting

POST `/api/v1/teams/{team_id}/rewrite` TEAM ADMIN+

Set a remote team name rewrite for an inbound connection.

Request Body

```
{
  "connection": "relay-from-a",
  "remote_team_name": "remote-team-alpha"
}
```

Validation

- Both `connection` and `remote_team_name` must be non-empty
- Connection must be inbound and linked to this team

Success Response - 200

```
{
  "status": "ok",
  "team_id": "abc123...",
  "connection": "relay-from-a",
  "remote_team_name": "remote-team-alpha"
}
```

Error Responses

Status	Meaning
`400`	Validation error, connection not inbound, or not linked
`403`	Insufficient permissions
`409`	Rewrite index already exists for this connection + remote_team_name pair on another team
`500`	Internal error

DELETE `/api/v1/teams/{team_id}/rewrite` TEAM ADMIN+

Clear a remote team name rewrite.

Query Parameters

`connection`	Required. Connection name to clear the rewrite for.

Success Response - 200

```
{
  "status": "ok",
  "team_id": "abc123...",
  "connection": "relay-from-a"
}
```

Connection Prompts

> **Note**
> These endpoints are called by interactive buttons in prompt messages. They are not intended to be called directly.

POST `/api/v1/prompt/accept` TEAM ADMIN+

Accept button on team-level inbound connection prompt.

Context

```
{"team_id": "...", "conn_name": "..."}
```

Links inbound connection to team, deletes the prompt, and updates the prompt post message to show acceptance. Returns a `PostActionIntegrationResponse`.

POST `/api/v1/prompt/block` TEAM ADMIN+

Block button on team-level inbound connection prompt.

Context

```
{"team_id": "...", "conn_name": "..."}
```

Sets prompt state to `blocked` and updates the prompt post message. Connection stays unlinked until `reset-prompt` is used.

POST `/api/v1/prompt/channel/accept` TEAM ADMIN+

Accept button on channel-level inbound connection prompt.

Context

```
{"channel_id": "...", "conn_name": "..."}
```

Links inbound connection to channel, deletes prompt, and updates the post.

POST `/api/v1/prompt/channel/block` TEAM ADMIN+

Block button on channel-level inbound connection prompt.

Context

```
{"channel_id": "...", "conn_name": "..."}
```

Sets channel prompt state to `blocked` and updates the post.

Connection Requests

> **Note**
> These endpoints are called by interactive buttons in connection request DMs sent to System Admins. They are not intended to be called directly. Requests are created when a Team Admin runs `/crossguard init-team` (or the REST equivalent) while the plugin is in request mode (`RestrictToSystemAdmins` + `AllowTeamAdminRequests` both enabled).

POST `/api/v1/request/approve` SYSTEM ADMIN

Approve a pending connection link request. Links the connection to the team, deletes the request, updates all admin DM posts, and notifies the requester.

Request Body

`PostActionIntegrationRequest` with context:

```
{"team_id": "...", "conn_key": "..."}
```

Behavior

- Executes the team init (same as a direct System Admin link)
- Removes Approve/Deny buttons from all admin DM posts for this request
- Sends DM to requester confirming approval
- If the connection was removed from config while pending, cancels the request and notifies the requester

Returns a `PostActionIntegrationResponse`.

POST `/api/v1/request/deny` SYSTEM ADMIN

Deny a pending connection link request. Opens an interactive dialog for the admin to provide an optional reason.

Request Body

`PostActionIntegrationRequest` with context:

```
{"team_id": "...", "conn_key": "..."}
```

Behavior

- Opens a dialog with an optional "Reason for denial" text area
- Dialog submission is handled by `/api/v1/request/deny-submit`

Returns a `PostActionIntegrationResponse`.

POST `/api/v1/request/deny-submit` SYSTEM ADMIN

Dialog submission handler for deny with optional reason.

Request Body

`SubmitDialogRequest` with:

- `state`: `team_id|conn_key`
- `submission.reason`: optional denial reason text

Behavior

- Deletes the pending request
- Removes Approve/Deny buttons from all admin DM posts for this request

- Sends DM to requester with denial and optional reason
- If the dialog was cancelled, does nothing

Returns `200` on success.

Channel Connection Requests

When both `RestrictToSystemAdmins` and `AllowChannelAdminRequests` are enabled, Channel Admins can submit channel-level connection link requests. These endpoints handle the Team Admin approval workflow. They mirror the team-level request endpoints but operate on channel connections with Team Admins as the approvers.

POST `/api/v1/channel-request/approve` TEAM ADMIN+

Approve a pending channel connection link request. Links the connection to the channel, deletes the request, updates all Team Admin DM posts, and notifies the requester.

Request Body

`PostActionIntegrationRequest` with context:

```
{"channel_id": "...", "team_id": "...", "conn_key": "..."}
```

Behavior

- Validates the connection still exists in config and the team still has it linked
- Validates the target channel still exists
- Executes the channel init (same as a direct Channel Admin link)
- Removes Approve/Deny buttons from all Team Admin DM posts for this request
- Sends DM to requester confirming approval
- If the connection was removed from config or unlinked from the team while pending, cancels the request and notifies the requester

Returns a `PostActionIntegrationResponse`.

POST `/api/v1/channel-request/deny` TEAM ADMIN+

Deny a pending channel connection link request. Opens an interactive dialog for the Team Admin to provide an optional reason.

Request Body

`PostActionIntegrationRequest` with context:

```
{"channel_id": "...", "team_id": "...", "conn_key": "..."}
```

Behavior

- Opens a dialog with an optional "Reason for denial" text area
- Dialog submission is handled by `/api/v1/channel-request/deny-submit`

Returns a `PostActionIntegrationResponse`.

POST `/api/v1/channel-request/deny-submit` TEAM ADMIN+

Dialog submission handler for channel request deny with optional reason.

Request Body

`SubmitDialogRequest` with:

- `state`: `channel_id|team_id|conn_key`
- `submission.reason`: optional denial reason text

Behavior

- Deletes the pending channel connection request
- Removes Approve/Deny buttons from all Team Admin DM posts for this request

- Sends DM to requester with denial and optional reason
- If the dialog was cancelled, does nothing

Returns `200` on success.

Dialog Handling

POST `/api/v1/dialog/select-connection` AUTHENTICATED

Interactive dialog handler when user selects a connection from the dropdown.

Request Body

`SubmitDialogRequest` with:

- `connection_name` field: selected connection in `direction:name` format
- `state` field: `action:target_id` (e.g., `init-team:abc123`, `teardown-channel:xyz789`)

Supported Actions

Action	Description
`init-team`	Link connection to team
`teardown-team`	Unlink connection from team
`init-channel`	Link connection to channel
`teardown-channel`	Unlink connection from channel

Validates permissions, performs the action, and sends an ephemeral message with the result. Returns `200` on success.

Slash Command Autocomplete

This endpoint backs the dynamic-list autocomplete arguments registered by `/crossguard` subcommands. Mattermost invokes it as the user types, passing `team_id`, `channel_id`, `user_id`, and `user_input` as query parameters. Callers without the permission to run the corresponding subcommand receive an empty list rather than a 403, so the autocomplete UI does not surface a permission error mid-typing.

GET `/api/v1/autocomplete/connections/{action}` AUTHENTICATED

Returns the list of connection names suitable for tab-completion of the named slash-command action.

Path Parameters

Action	List Returned
`init-team`	All configured connections.
`init-channel`	Connections currently linked to the team.
`teardown-team`	Connections currently linked to the team.
`teardown-channel`	Connections currently linked to the channel.

Query Parameters

- `team_id`: Required for all actions. The team context where the slash command was typed.
- `channel_id`: Required for channel actions.
- `user_id`, `user_input`: Passed by Mattermost; the handler uses the `Mattermost-User-Id` header for the permission check.

Response

JSON array of `AutocompleteListItem` objects, each carrying an `Item` (the connection key in `direction:name` form) and a `HelpText` describing the direction. Empty list returned when the caller lacks permission, the action is unknown, or no connections match.

Message Transport Interface

Cross Guard supports NATS, Azure Queue Storage, Azure Blob Storage, and Azure Service Bus as transport layers for relaying messages between Mattermost instances. For the complete wire protocol reference, including all message envelope schemas, payload definitions, connection configuration, data flow diagrams, and full wire format examples, see the dedicated Transport Interface Reference.

Error Codes

Reference for every numeric `error_code` emitted by Cross Guard logs

How to Read This Page

Every operator-facing log call (LogInfo, LogWarn, LogError) in non-test code carries a unique numeric `error_code` as its first key-value pair, sourced from `server/errcode/codes.go`. The integer value is the stable contract for log grep across plugin versions: the Go constant name may be renamed, but the number never changes once assigned.

Example log line:

```
{"level":"warn","msg":"...","error_code":15301,"epoch":"epoch01...","sequence":42}
```

Codes are allocated per source file in non-overlapping 1000-wide ranges. The first two digits identify the subsystem. Each table row below lists the constant name, the integer value, the inferred log level (where the inline comment in `codes.go` declares one), a description, and operator-facing troubleshooting guidance.

> **Generated file**
> This page is regenerated from `server/errcode/codes.go` and `scripts/generate-error-codes/annotations.yaml` by running `make generate-error-codes`. Hand edits will be overwritten; add descriptions and troubleshooting tips to the annotations YAML instead.

Range Quick Reference

Range	Source file	Count
`10000-10999`	hooks.go	25
`11000-11999`	api.go	17

Range	Source file	Count
`12000-12999`	configuration.go	3
`13000-13999`	command.go	1
`14000-14999`	service.go	39
`15000-15999`	inbound.go	37
`16000-16999`	connections.go	6
`18000-18999`	azure_blob_provider.go	50
`19000-19999`	azure_provider.go	11
`20000-20999`	nats_provider.go	5
`21000-21999`	prompt.go	17
`23000-23999`	store/caching.go	1
`24000-24999`	request.go	17
`25000-25999`	channel_request.go	18
`26000-26999`	azure_servicebus_provider.go	8
`27000-27999`	plugin.go	12
`28000-28999`	inbound_files.go	15
`29000-29999`	wire/*	7

Total: 289 codes across 18 source-file blocks.

hooks.go (10000-10999)

Emitted from the shared-channels framework hooks (`OnSharedChannelsSyncMsg`, `OnSharedChannelsAttachmentSyncMsg`, `OnSharedChannelsProfileImageSyncMsg`, `OnSharedChannelsPing`) in `server/hooks.go`. These run for every sync event the framework hands the plugin, so transient failures here can prevent the relay of individual sync cycles.

Code	Name / Level	Description	Troubleshooting
`10100`	`OutboundSyncMsgNoRemoteMatch`	*(no description)*	*(see source)*
`10101`	`OutboundSyncMsgChannelLookupFailed`	*(no description)*	*(see source)*
`10102`	`OutboundSyncMsgPublishFailed`	*(no description)*	*(see source)*
`10103`	`OutboundAttachmentStubbed`	*(no description)*	*(see source)*
`10104`	`OutboundProfileImageStubbed`	*(no description)*	*(see source)*
`10105`	`PingNoRemoteMatch`	*(no description)*	*(see source)*
`10106`	`PingOutboundUnhealthy`	*(no description)*	*(see source)*
`10107`	`PingInboundUnhealthy`	*(no description)*	*(see source)*
`10108`	`OutboundSyncMsgUserLookupFailed`	*(no description)*	*(see source)*
`10109`	`OutboundSyncMsgUsersAugmented`	*(no description)*	*(see source)*
`10110`	`OutboundAttachmentNoOutboundProvider`	*(no description)*	*(see source)*
`10111`	`OutboundAttachmentConfigParseFailed`	*(no description)*	*(see source)*
`10112`	`OutboundAttachmentDisabled`	*(no description)*	*(see source)*
`10113`	`OutboundAttachmentFiltered`	*(no description)*	*(see source)*
`10114`	`OutboundAttachmentSizeExceeded`	*(no description)*	*(see source)*
`10115`	`OutboundAttachmentFileFetchFailed`	*(no description)*	*(see source)*
`10116`	`OutboundAttachmentChannelLookupFailed`	*(no description)*	*(see source)*

Code	Name / Level	Description	Troubleshooting
`10117`	`OutboundAttachmentTeamLookupFailed`	*(no description)*	*(see source)*
`10118`	`OutboundAttachmentFileInfoEncodeFail`	*(no description)*	*(see source)*
`10119`	`OutboundAttachmentUploadFailed`	*(no description)*	*(see source)*
`10130`	`OutboundProfileImageNoOutboundProvider`	*(no description)*	*(see source)*
`10131`	`OutboundProfileImageConfigParseFailed`	*(no description)*	*(see source)*
`10132`	`OutboundProfileImageDisabled`	*(no description)*	*(see source)*
`10133`	`OutboundProfileImageFetchFailed`	*(no description)*	*(see source)*
`10134`	`OutboundProfileImageUploadFailed`	*(no description)*	*(see source)*

api.go (11000-11999)

Emitted from REST API handlers in `server/api.go`. Most codes correspond to a System Console action (test connection, set rewrite) or to a slash-command action dispatched via the API.

Code	Name / Level	Description	Troubleshooting
`11000`	`APINATSTestConnectFailed`	*(no description)*	*(see source)*
`11001`	`APIBuildTestMessageFailed`	*(no description)*	*(see source)*
`11002`	`APIPublishTestMessageFailed`	*(no description)*	*(see source)*
`11003`	`APIFlushTestMessageFailed`	*(no description)*	*(see source)*
`11004`	`APITestMessageSent`	*(no description)*	*(see source)*
`11005`	`APISubscribeInboundTestFailed`	*(no description)*	*(see source)*
`11006`	`APIFlushNATSConnFailed`	*(no description)*	*(see source)*

Code	Name / Level	Description	Troubleshooting
11007	APIInboundTestSubscribeOK	*(no description)*	*(see source)*
11008	APIAzureQueueTestFailed	*(no description)*	*(see source)*
11009	APIAzureBlobTestFailed	*(no description)*	*(see source)*
11010	APIGetUserFailed	*(no description)*	*(see source)*
11011	APIInitChannelGetTeamConns	*(no description)*	*(see source)*
11012	APITeardownChannelGetChanConns	*(no description)*	*(see source)*
11013	APITeardownTeamGetTeamConns	*(no description)*	*(see source)*
11014	APIBulkChannelConnectionsFailed	*(no description)*	*(see source)*
11015	APITeamRewriteSet	*(no description)*	*(see source)*
11016	APITeamRewriteCleared	*(no description)*	*(see source)*

configuration.go (12000-12999)

Emitted at plugin configuration load and validation (`server/configuration.go`). Includes the Azure Service Principal credential-construction audit (`AzureSPAuditConstructed`) that records tenant ID, client ID, Azure cloud, and a short hash of the SP secret on every successful credential construction.

Code	Name / Level	Description	Troubleshooting
12000	ConfigSameConfigPassed	*(no description)*	*(see source)*

Code	Name / Level	Description	Troubleshooting
`12001`	`ConfigValidationWarn`	*(no description)*	*(see source)*
`12010`	`AzureSPAuditConstructed`	Emitted once per successful Azure Service Principal credential construction across the three Azure providers. The corresponding INFO log records `tenant_id`, `client_id`, `azure_cloud`, and the first 12 hex chars of SHA-256 of the secret so operators can correlate "which SP / secret was active" without exposing the secret value itself.	Informational. Each appearance corresponds to a credential construction (plugin start, configuration change, or SP rotation picked up at the next load). Use it as the audit trail for SP rotations. The secret value itself is never logged.

command.go (13000-13999)

Emitted from slash-command execution (`server/command.go`).

Code	Name / Level	Description	Troubleshooting
`13000`	`CommandOpenConnDialogFailed`	*(no description)*	*(see source)*

service.go (14000-14999)

Emitted from the team and channel initialization service (`server/service.go`). Most failures here mean a KV write or read error during a link or teardown action.

Code	Name / Level	Description	Troubleshooting
`14000`	`ServiceInitTeamGetConnsFailed`	*(no description)*	*(see source)*
`14001`	`ServiceAddTeamConnFailed`	*(no description)*	*(see source)*

Code	Name / Level	Description	Troubleshooting
14002	ServiceInitTeamReReadConnsFailed	*(no description)*	*(see source)*
14003	ServiceAddTeamInitializedFailed	*(no description)*	*(see source)*
14004	ServicePostTeamInitMsgFailed	*(no description)*	*(see source)*
14005	ServiceCheckTeamStatusFailed	*(no description)*	*(see source)*
14006	ServiceGetInitializedTeamsFailed	*(no description)*	*(see source)*
14007	ServiceTeamStatusLookupTeamFailed	*(no description)*	*(see source)*
14008	ServiceTeamStatusGetConnsFailed	*(no description)*	*(see source)*
14009	ServiceParseOutboundConnFailed	*(no description)*	*(see source)*
14010	ServiceParseInboundConnFailed	*(no description)*	*(see source)*
14011	ServiceTeardownGetChanConnsFailed	*(no description)*	*(see source)*
14012	ServiceTeardownGetTeamConnsFailed	*(no description)*	*(see source)*
14013	ServiceInitChanGetTeamConnsFailed	*(no description)*	*(see source)*
14014	ServiceInitChanGetChanConnsFailed	*(no description)*	*(see source)*
14015	ServiceAddChanConnFailed	*(no description)*	*(see source)*
14016	ServiceInitChanReReadConnsFailed	*(no description)*	*(see source)*
14017	ServiceChanHeaderPrefixFailed	*(no description)*	*(see source)*
14018	ServicePostChanInitMsgFailed	*(no description)*	*(see source)*
14019	ServiceRemoveChanGetConnsFailed	*(no description)*	*(see source)*
14020	ServiceRemoveChanConnFailed	*(no description)*	*(see source)*
14021	ServiceRemoveChanReReadConnsFailed	*(no description)*	*(see source)*

Code	Name / Level	Description	Troubleshooting
14022	ServiceDeleteChanConnsFailed	*(no description)*	*(see source)*
14023	ServiceChanHeaderRemovePrefixFailed	*(no description)*	*(see source)*
14024	ServicePostChanTeardownMsgFailed	*(no description)*	*(see source)*
14025	ServiceTeardownTeamGetConnsFailed	*(no description)*	*(see source)*
14026	ServiceRemoveTeamConnFailed	*(no description)*	*(see source)*
14027	ServiceTeardownTeamReReadConnsFailed	*(no description)*	*(see source)*
14028	ServiceRemoveTeamInitializedFailed	*(no description)*	*(see source)*
14029	ServicePostTeamTeardownMsgFailed	*(no description)*	*(see source)*
14030	ServiceGlobalParseOutConnFailed	*(no description)*	*(see source)*
14031	ServiceGlobalParseInConnFailed	*(no description)*	*(see source)*
14032	ServiceMapParseOutConnFailed	*(no description)*	*(see source)*
14033	ServiceMapParseInConnFailed	*(no description)*	*(see source)*
14100	ServiceShareChannelFailed	*(no description)*	*(see source)*
14101	ServiceInviteRemoteFailed	*(no description)*	*(see source)*
14102	ServiceUninviteRemoteFailed	*(no description)*	*(see source)*
14103	ServiceUnshareFailed	*(no description)*	*(see source)*
14104	ServiceShareForRemoteRollback	*(no description)*	*(see source)*

inbound.go (15000-15999)

Emitted from the inbound message pipeline (`server/inbound.go`), including the single-active-receiver election (`InboundActive*`, 15200-range), the inbound sequencer (`InboundSeq*`, 15300-range), and the cursor checkpoint ticker (15400-range). These are the operator-facing audit codes for envelope ordering and clustered failover.

Code	Name / Level	Description	Troubleshooting
`15000`	`InboundParseConnsFailed`	*(no description)*	*(see source)*
`15001`	`InboundConnectFailed`	*(no description)*	*(see source)*
`15002`	`InboundSubscribeFailed`	*(no description)*	*(see source)*
`15003`	`InboundSubscriptionEstablished`	*(no description)*	*(see source)*
`15005`	`InboundUnmarshalFailed`	*(no description)*	*(see source)*
`15100`	`InboundReceiveSyncFailed`	*(no description)*	*(see source)*
`15101`	`InboundPostSyncErrors`	*(no description)*	*(see source)*
`15102`	`InboundAttachmentStubbed`	*(no description)*	*(see source)*
`15103`	`InboundProfileImageStubbed`	*(no description)*	*(see source)*
`15104`	`InboundUnknownType`	*(no description)*	*(see source)*
`15105`	`InboundNoRemoteForConn`	*(no description)*	*(see source)*

Code	Name / Level	Description	Troubleshooting
15106	InboundNotConfigured	*(no description)*	*(see source)*
15107	InboundTestReceived	*(no description)*	*(see source)*
15200	InboundActiveNodeElected	Single-active-receiver election (Phase 2).	*(see source)*
15201	InboundActiveNodeSteppedDown	*(no description)*	*(see source)*
15202	InboundActiveLeaseRenewalFailed	*(no description)*	*(see source)*
15203	InboundActiveLeaseLost	*(no description)*	*(see source)*
15204	InboundActiveLeaseAcquireFailed	*(no description)*	*(see source)*
15205	InboundActiveStepdownEventPubErr	*(no description)*	*(see source)*
15206	InboundActiveStepdownEventRecv	*(no description)*	*(see source)*
15207	InboundActiveSubscribeFailed	*(no description)*	*(see source)*
15208	InboundActiveUnsubscribed	*(no description)*	*(see source)*
15300	InboundSeqInOrder	Sequencer state machine (Phase 4).	*(see source)*
15301	InboundSeqGapDetected	*(no description)*	*(see source)*

Code	Name / Level	Description	Troubleshooting
15302	InboundSeqGapFilled	*(no description)*	*(see source)*
15303	InboundSeqGapTimeout	*(no description)*	*(see source)*
15304	InboundSeqBufferOverflow	*(no description)*	*(see source)*
15305	InboundSeqDuplicate	*(no description)*	*(see source)*
15306	InboundSeqEpochReset	*(no description)*	*(see source)*
15307	InboundSeqStaleEpoch	*(no description)*	*(see source)*
15308	InboundSeqMissingFields	*(no description)*	*(see source)*
15309	InboundSeqTestEpochMatch	*(no description)*	*(see source)*
15310	InboundSeqTestEpochChange	*(no description)*	*(see source)*
15400	InboundSeqCheckpointWritten	Cursor checkpointing (Phase 5).	*(see source)*
15401	InboundSeqCheckpointLoaded	*(no description)*	*(see source)*
15402	InboundSeqCheckpointLoadFailed	*(no description)*	*(see source)*
15403	InboundSeqCheckpointStale	*(no description)*	*(see source)*

connections.go (16000-16999)

Emitted from the outbound publisher and the file-upload path (`server/connections.go`). Covers transport publish failures and file-upload errors.

Code	Name / Level	Description	Troubleshooting
`16000`	`ConnectionsParseOutboundFailed`	*(no description)*	*(see source)*
`16001`	`ConnectionsConnectOutboundFailed`	*(no description)*	*(see source)*
`16002`	`ConnectionsOutboundEstablished`	*(no description)*	*(see source)*
`16004`	`ConnectionsMessageSplit`	*(no description)*	*(see source)*
`16005`	`ConnectionsSerializePartFailed`	*(no description)*	*(see source)*
`16006`	`ConnectionsPublishPartFailed`	*(no description)*	*(see source)*

azure_blob_provider.go (18000-18999)

Emitted from the Azure Blob Storage provider, including the batch WAL sealer, the lease-based reader, and the deferred file blob handler (`server/azure_blob_provider.go`).

Code	Name / Level	Description	Troubleshooting
`18000`	`AzureBlobContainerCreateRetry`	*(no description)*	*(see source)*
`18001`	`AzureBlobWALInTempStorage`	*(no description)*	*(see source)*
`18002`	`AzureBlobWALDirFsyn`	*(no description)*	*(see source)*

Code	Name / Level	Description	Troubleshooting
	`cFailed`		
`18003`	`AzureBlobMarshalPendingFailed`	*(no description)*	*(see source)*
`18004`	`AzureBlobWriteCompanionFailed`	*(no description)*	*(see source)*
`18005`	`AzureBlobShutdownLeftWALFiles`	*(no description)*	*(see source)*
`18006`	`AzureBlobWALFsyncRotationFailed`	*(no description)*	*(see source)*
`18007`	`AzureBlobWALCloseRotationFailed`	*(no description)*	*(see source)*
`18008`	`AzureBlobWALUploadFailed`	*(no description)*	*(see source)*
`18009`	`AzureBlobDeleteWALFailed`	*(no description)*	*(see source)*
`18010`	`AzureBlobMarshalFai`	*(no description)*	*(see source)*

Code	Name / Level	Description	Troubleshooting
	`ledRefsFailed`		
`18011`	`AzureBlobRewriteCompanionFailed`	*(no description)*	*(see source)*
`18012`	`AzureBlobDeleteCompanionFailed`	*(no description)*	*(see source)*
`18013`	`AzureBlobDeferredFileFetchFailed`	*(no description)*	*(see source)*
`18014`	`AzureBlobDeferredFileUploadFailed`	*(no description)*	*(see source)*
`18015`	`AzureBlobShutdownMarshalResidualFailed`	*(no description)*	*(see source)*
`18016`	`AzureBlobShutdownPersistResidualFailed`	*(no description)*	*(see source)*
`18017`	`AzureBlobShutdownPersistedResidual`	*(no description)*	*(see source)*

Code	Name / Level	Description	Troubleshooting
`18018`	`AzureBlobListFailed`	*(no description)*	*(see source)*
`18019`	`AzureBlobDeleteRetryFailed`	*(no description)*	*(see source)*
`18020`	`AzureBlobDownloadFailed`	*(no description)*	*(see source)*
`18021`	`AzureBlobHandlerError`	*(no description)*	*(see source)*
`18022`	`AzureBlobDeleteFailed`	*(no description)*	*(see source)*
`18023`	`AzureBlobWriteProcessedMarkerFailed`	*(no description)*	*(see source)*
`18024`	`AzureBlobClearProcessedMarkerFailed`	*(no description)*	*(see source)*
`18025`	`AzureBlobLockGetFailed`	*(no description)*	*(see source)*
`18026`	`AzureBlobLockTokenGenFailed`	*(no description)*	*(see source)*

Code	Name / Level	Description	Troubleshooting
`18027`	`AzureBlobLockSetFailed`	*(no description)*	*(see source)*
`18028`	`AzureBlobCorruptLockReclaimFailed`	*(no description)*	*(see source)*
`18029`	`AzureBlobStaleLockReclaimFailed`	*(no description)*	*(see source)*
`18030`	`AzureBlobLockReleaseFailed`	*(no description)*	*(see source)*
`18031`	`AzureBlobLockReleaseGetFailed`	*(no description)*	*(see source)*
`18032`	`AzureBlobLockReleaseCorrupt`	*(no description)*	*(see source)*
`18033`	`AzureBlobLockReleaseReclaimedByOther`	*(no description)*	*(see source)*
`18034`	`AzureBlobLockReleaseConditionalFailed`	*(no description)*	*(see source)*

Code	Name / Level	Description	Troubleshooting
`18035`	`AzureBlobWALRecoveryScanRootFailed`	*(no description)*	*(see source)*
`18036`	`AzureBlobWALRecoveryScanDirFailed`	*(no description)*	*(see source)*
`18037`	`AzureBlobWALRecoverySkipUnrecognized`	*(no description)*	*(see source)*
`18038`	`AzureBlobWALRecoveryUploadFailed`	*(no description)*	*(see source)*
`18039`	`AzureBlobWALRecoveryDeleteFailed`	*(no description)*	*(see source)*
`18040`	`AzureBlobWALRecoveryReadCompanionFail`	*(no description)*	*(see source)*
`18041`	`AzureBlobWALRecoveryMalformedCompanion`	*(no description)*	*(see source)*
`18042`	`AzureBlobWALRecover`	*(no description)*	*(see source)*

Code	Name / Level	Description	Troubleshooting
	`yMarshalRemaining`		
`18043`	`AzureBlobWALRecoveryRewriteCompanion`	*(no description)*	*(see source)*
`18044`	`AzureBlobFileListFailed`	*(no description)*	*(see source)*
`18045`	`AzureBlobFileDeleteRetryFailed`	*(no description)*	*(see source)*
`18046`	`AzureBlobFileDownloadFailed`	*(no description)*	*(see source)*
`18047`	`AzureBlobFileHandlerError`	*(no description)*	*(see source)*
`18048`	`AzureBlobFileDeleteFailed`	*(no description)*	*(see source)*
`18049`	`AzureBlobSPCredentialFailed`	AzureBlobSPCredentialFailed is emitted when constructing the azidentity ClientSecretCredential for the Azure Blob provider in service-principal mode fails (e.g., malformed tenant_id).	*(see source)*

azure_provider.go (19000-19999)

Emitted from the Azure Queue Storage provider (`server/azure_provider.go`). Covers queue dequeue, blob container operations for file transfer, and credential construction (shared key and Service Principal).

Code	Name / Level	Description	Troubleshooting
`19000`	`AzureQueueCreateQueueFailed`	*(no description)*	*(see source)*
`19001`	`AzureQueueCreateContainerFail`	*(no description)*	*(see source)*
`19002`	`AzureQueueDequeueFailed`	*(no description)*	*(see source)*
`19003`	`AzureQueueDecodeFailed`	*(no description)*	*(see source)*
`19004`	`AzureQueueHandlerRetry`	*(no description)*	*(see source)*
`19005`	`AzureQueueDeleteProcessedFail`	*(no description)*	*(see source)*
`19006`	`AzureQueueBlobListFailed`	*(no description)*	*(see source)*

Code	Name / Level	Description	Troubleshooting
`19007`	`AzureQueueBlobDownloadFailed`	*(no description)*	*(see source)*
`19008`	`AzureQueueBlobHandlerError`	*(no description)*	*(see source)*
`19009`	`AzureQueueBlobDeleteFailed`	*(no description)*	*(see source)*
`19010`	`AzureQueueSPCredentialFailed`	AzureQueueSPCredentialFailed is emitted when constructing the azidentity ClientSecretCredential for the Azure Queue provider in service-principal mode fails.	*(see source)*

nats_provider.go (20000-20999)

Emitted from the NATS transport provider (`server/nats_provider.go`). Covers connection lifecycle and JetStream Object Store operations for file transfer.

Code	Name / Level	Description	Troubleshooting
`20000`	`NATSDownloadFileFailed`	*(no description)*	*(see source)*
`20001`	`NATSFileHandlerError`	*(no description)*	*(see source)*
`20002`	`NATSDisconnected`	*(no description)*	*(see source)*
`20003`	`NATSReconnected`	*(no description)*	*(see source)*
`20004`	`NATSDeleteFileFailed`	*(no description)*	*(see source)*

prompt.go (21000-21999)

Emitted from the inbound-connection acceptance prompt flow (`server/prompt.go`). Covers the Accept/Block interactive prompts and their lifecycle.

Code	Name / Level	Description	Troubleshooting
`21000`	`PromptGetConnPromptFailed`	*(no description)*	*(see source)*
`21001`	`PromptGetTownSquareFailed`	*(no description)*	*(see source)*
`21002`	`PromptCreatePostFailed`	*(no description)*	*(see source)*
`21003`	`PromptSavePromptFailed`	*(no description)*	*(see source)*
`21004`	`PromptAcceptGetPromptFailed`	*(no description)*	*(see source)*
`21005`	`PromptDeletePromptFailed`	*(no description)*	*(see source)*
`21006`	`PromptBlockGetPromptFailed`	*(no description)*	*(see source)*
`21007`	`PromptSetBlockedFailed`	*(no description)*	*(see source)*
`21008`	`PromptGetChanPromptFailed`	*(no description)*	*(see source)*
`21009`	`PromptCreateChanPostFailed`	*(no description)*	*(see source)*
`21010`	`PromptSaveChanPromptFailed`	*(no description)*	*(see source)*
`21011`	`PromptChanAcceptGetFailed`	*(no description)*	*(see source)*
`21012`	`PromptDeleteChanPromptFailed`	*(no description)*	*(see source)*
`21013`	`PromptChanBlockGetFailed`	*(no description)*	*(see source)*
`21014`	`PromptSetChanBlockedFailed`	*(no description)*	*(see source)*
`21015`	`PromptGetPostForUpdateFailed`	*(no description)*	*(see source)*
`21016`	`PromptUpdatePostFailed`	*(no description)*	*(see source)*

store/caching.go (23000-23999)

Emitted from the KV cache layer (`server/store/caching.go`). The cache is cluster-aware via `PluginClusterEvent` invalidation; failures here are usually transient.

Code	Name / Level	Description	Troubleshooting
`23000`	`StoreCachePublishInvalidationFailed`	*(no description)*	*(see source)*

request.go (24000-24999)

Emitted from the team-admin connection request workflow (`server/request.go`). Active when `RestrictToSystemAdmins` and `AllowTeamAdminRequests` are both enabled.

Code	Name / Level	Description	Troubleshooting
`24000`	`RequestGetFailed`	*(no description)*	*(see source)*
`24002`	`RequestGetDMChannelFailed`	*(no description)*	*(see source)*
`24003`	`RequestCreateDMPostFailed`	*(no description)*	*(see source)*
`24004`	`RequestSaveFailed`	*(no description)*	*(see source)*
`24005`	`RequestApproveGetFailed`	*(no description)*	*(see source)*
`24006`	`RequestApproveExecFailed`	*(no description)*	*(see source)*
`24007`	`RequestApproveDeleteFailed`	*(no description)*	*(see source)*
`24008`	`RequestApproveNotifyFailed`	*(no description)*	*(see source)*
`24009`	`RequestDenyDialogFailed`	*(no description)*	*(see source)*
`24010`	`RequestDenyGetFailed`	*(no description)*	*(see source)*
`24011`	`RequestDenyDeleteFailed`	*(no description)*	*(see source)*

Code	Name / Level	Description	Troubleshooting
`24012`	`RequestDenyNotifyFailed`	*(no description)*	*(see source)*
`24013`	`RequestUpdatePostFailed`	*(no description)*	*(see source)*
`24014`	`RequestNoSystemAdmins`	*(no description)*	*(see source)*
`24015`	`RequestConnRemovedFromConfig`	*(no description)*	*(see source)*
`24016`	`RequestConfirmDMFailed`	*(no description)*	*(see source)*
`24017`	`RequestNoAdminsNotified`	*(no description)*	*(see source)*

channel_request.go (25000-25999)

Emitted from the channel-admin connection request workflow (`server/channel_request.go`). Active when `RestrictToSystemAdmins` and `AllowChannelAdminRequests` are both enabled.

Code	Name / Level	Description	Troubleshooting
`25000`	`ChanRequestGetFailed`	*(no description)*	*(see source)*
`25002`	`ChanRequestGetDMChannelFailed`	*(no description)*	*(see source)*
`25003`	`ChanRequestCreateDMPostFailed`	*(no description)*	*(see source)*
`25004`	`ChanRequestSaveFailed`	*(no description)*	*(see source)*
`25005`	`ChanRequestApproveGetFailed`	*(no description)*	*(see source)*
`25006`	`ChanRequestApproveExecFailed`	*(no description)*	*(see source)*
`25007`	`ChanRequestApproveDeleteFailed`	*(no description)*	*(see source)*
`25008`	`ChanRequestApproveNotifyFailed`	*(no description)*	*(see source)*
`25009`	`ChanRequestDenyDialogFailed`	*(no description)*	*(see source)*

Code	Name / Level	Description	Troubleshooting
25010	ChanRequestDenyGetFailed	*(no description)*	*(see source)*
25011	ChanRequestDenyDeleteFailed	*(no description)*	*(see source)*
25012	ChanRequestDenyNotifyFailed	*(no description)*	*(see source)*
25013	ChanRequestUpdatePostFailed	*(no description)*	*(see source)*
25014	ChanRequestNoTeamAdmins	*(no description)*	*(see source)*
25015	ChanRequestConnRemovedFromConfig	*(no description)*	*(see source)*
25016	ChanRequestConfirmDMFailed	*(no description)*	*(see source)*
25017	ChanRequestNoAdminsNotified	*(no description)*	*(see source)*
25018	ChanRequestGetTeamAdminsFailed	*(no description)*	*(see source)*

azure_servicebus_provider.go (26000-26999)

Emitted from the Azure Service Bus provider (`server/azure_servicebus_provider.go`). Covers the PeekLock receiver, AMQP send path, blob sidecar handling, and credential construction (SAS connection string and Service Principal).

Code	Name / Level	Description	Troubleshooting
26000	ServiceBusSendFailed	*(no description)*	*(see source)*
26001	ServiceBusReceiveFailed	*(no description)*	*(see source)*

Code	Name / Level	Description	Troubleshooting
`26002`	`ServiceBusCompleteFailed`	*(no description)*	*(see source)*
`26003`	`ServiceBusAbandonFailed`	*(no description)*	*(see source)*
`26004`	`ServiceBusRedelivery`	*(no description)*	*(see source)*
`26005`	`ServiceBusMalformedBody`	*(no description)*	*(see source)*
`26006`	`APIAzureServiceBusTestFailed`	*(no description)*	*(see source)*
`26007`	`ServiceBusSPCredentialFailed`	ServiceBusSPCredentialFailed is emitted when constructing the azidentity ClientSecretCredential for the Azure Service Bus provider in service-principal mode fails.	*(see source)*

plugin.go (27000-27999)

Emitted from plugin lifecycle hooks (`server/plugin.go`): activation, deactivation, framework registration, and the envelope-archive configuration on `OnActivate`.

Code	Name / Level	Description	Troubleshooting
`27000`	`PluginRegisterFailed`	*(no description)*	*(see source)*

Code	Name / Level	Description	Troubleshooting
`27001`	`PluginUnregisterFailed`	*(no description)*	*(see source)*
`27002`	`PluginEpochAssigned`	*(no description)*	*(see source)*
`27100`	`PluginMigrateListKVFailed`	Upgrade migrations (one-shot, run from OnActivate).	*(see source)*
`27101`	`PluginMigrateDeleteKVFailed`	*(no description)*	*(see source)*
`27102`	`PluginMigrateRecordKVFailed`	*(no description)*	*(see source)*
`27103`	`PluginMigrateGetChannelFailed`	*(no description)*	*(see source)*
`27104`	`PluginMigrateShareFailed`	*(no description)*	*(see source)*
`27105`	`PluginMigrateInviteFailed`	*(no description)*	*(see source)*
`27106`	`PluginMigrateSummary`	*(no description)*	*(see source)*
`27107`	`PluginMigrateDeferred`	*(no description)*	*(see source)*
`27108`	`PluginEnvelopeArchiveConfig`	*(no description)*	*(see source)*

inbound_files.go (28000-28999)

Emitted from the inbound file-channel handler (`server/inbound_files.go`), which dispatches `WatchFiles` events to the framework's attachment and profile-image upsert APIs.

Code	Name / Level	Description	Troubleshooting
28000	InboundFileUnknownKind	*(no description)*	*(see source)*
28001	InboundAttachmentMissingHeader	*(no description)*	*(see source)*
28002	InboundAttachmentFileInfoDecodeFailed	*(no description)*	*(see source)*
28003	InboundAttachmentTeamLookupFailed	*(no description)*	*(see source)*
28004	InboundAttachmentChannelLookupFailed	*(no description)*	*(see source)*
28005	InboundAttachmentChannelUnlinked	*(no description)*	*(see source)*
28006	InboundAttachmentNoRemoteForConn	*(no description)*	*(see source)*
28007	InboundAttachmentReceiveFailed	*(no description)*	*(see source)*
28008	InboundProfileImageMissingHeader	*(no description)*	*(see source)*
28009	InboundProfileImageNoRemoteForConn	*(no description)*	*(see source)*
28010	InboundProfileImageReceiveFailed	*(no description)*	*(see source)*
28011	InboundFileWatcherRestart	*(no description)*	*(see source)*
28012	InboundAttachmentDisabled	*(no description)*	*(see source)*
28013	InboundAttachmentFiltered	*(no description)*	*(see source)*
28014	InboundProfileImageDisabled	*(no description)*	*(see source)*

wire/* (29000-29999)

Emitted by validators in `server/wire/` (via the `WireLogger` interface in `server/wire/validate.go`). These run during outbound marshalling to enforce the typed Props whitelists and the username resolution ladder; they are the source of truth for which content was dropped at the boundary.

Code	Name / Level	Description	Troubleshooting
`29000`	`WireUserDroppedNonConformingUsername` **ERROR**	Username failed validation and could not be recovered via the resolve-then-truncate ladder.	*(see source)*
`29001`	`WireUserDroppedNonConformingEmail` **ERROR**	Email failed validation; whole user record dropped.	*(see source)*
`29002`	`WireUsernameTruncatedAtColon` **WARN**	Colon-bearing Username sanitized by truncation (ladder step 3); RemoteUsername prop missing or itself non-conforming.	*(see source)*
`29003`	`WireUserPropDroppedNonConforming` **WARN**	Individual UserProps key (RemoteUsername or RemoteEmail) failed validation; prop dropped, user record retained.	*(see source)*
`29004`	`WirePostDroppedNonConforming` **ERROR**	reserved for symmetric Post validation coverage; not currently used.	*(see source)*
`29005`	`WireReactionDroppedNonConforming` **ERROR**	reserved for symmetric Reaction validation coverage; not currently used.	*(see source)*
`29006`	`WireUserDroppedAuditAtCaller` **ERROR**	emitted by buildOutboundEnvelopes when UserFromModel returns nil, correlating the drop with the carrying post.	*(see source)*